

Module 8: Machine Learning I

Learning Objectives

- Understand fundamental machine learning concepts and terminology
- Learn about different types of machine learning problems
- Master data preprocessing and feature engineering techniques
- Implement train-test split and cross-validation strategies
- Get familiar with scikit-learn library
- Build and evaluate linear regression models
- Build and evaluate logistic regression models
- Understand decision tree algorithms
- Learn comprehensive model evaluation metrics
- Recognize and handle overfitting and underfitting
- Apply various feature selection techniques

Topics Covered

1. Introduction to machine learning
2. Types of machine learning problems
3. Data preprocessing and feature engineering
4. Train-test split and cross-validation
5. Introduction to scikit-learn
6. Linear regression
7. Logistic regression
8. Decision trees
9. Model evaluation metrics
10. Overfitting and underfitting
11. Feature selection techniques

Prerequisites

- Basic Python programming
- NumPy and Pandas
- Matplotlib for visualization
- Understanding of statistics concepts

Exercises

- Hands-on implementation of ML algorithms
- Model evaluation and validation exercises
- Feature engineering challenges

- Overfitting detection and prevention

1. Introduction to Machine Learning

```
In [ ]: # Import all necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score, KF
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor, p
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, mean_squared_error, mean_abs
                             r2_score, classification_report, confusion_mat
                             precision_score, recall_score, f1_score, roc_a
from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEnco
from sklearn.impute import SimpleImputer
from sklearn.feature_selection import SelectKBest, f_classif, RFE, Select
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.datasets import make_classification, make_regression, load_b
import warnings
warnings.filterwarnings('ignore')

# Set style for better plots
plt.style.use('seaborn-v0_8')
sns.set_palette("husl")

print("All libraries imported successfully!")
print("Ready to start Machine Learning!")

# =====
# 1. INTRODUCTION TO MACHINE LEARNING
# =====

print("MACHINE LEARNING TYPES")
print("=" * 50)

print("\n1. SUPERVISED LEARNING")
print("    • Uses labeled training data (input-output pairs)")
print("    • Goal: Learn mapping from inputs to outputs")
print("    • Examples:")
print("        - Email spam detection (spam/not spam)")
print("        - House price prediction")
print("        - Medical diagnosis")
print("        - Image classification")

print("\n2. UNSUPERVISED LEARNING")
print("    • Uses unlabeled data")
print("    • Goal: Find hidden patterns in data")
print("    • Examples:")
print("        - Customer segmentation")
print("        - Anomaly detection")
print("        - Market basket analysis")
print("        - Dimensionality reduction")

print("\n3. REINFORCEMENT LEARNING")
```

```

print("    • Learns through interaction with environment")
print("    • Goal: Learn optimal actions through rewards")
print("    • Examples:")
print("        - Game playing (AlphaGo, Chess)")
print("        - Autonomous vehicles")
print("        - Trading algorithms")
print("        - Robotics")

print("\nCOMMON ML PROBLEMS")
print("=" * 30)
print("Classification: Predicting categories")
print("    Example: Spam detection, image recognition")
print("\nRegression: Predicting continuous values")
print("    Example: House prices, stock prices")
print("\nClustering: Grouping similar data points")
print("    Example: Customer segmentation, gene sequencing")
print("\nDimensionality Reduction: Reducing features")
print("    Example: PCA, t-SNE for visualization")

print("\nTYPICAL ML WORKFLOW")
print("=" * 25)
print("1. Data Collection")
print("2. Data Preprocessing")
print("3. Feature Engineering")
print("4. Model Selection")
print("5. Training")
print("6. Evaluation")
print("7. Deployment")
print("8. Monitoring & Maintenance")

# Create a visual representation
fig, ax = plt.subplots(1, 1, figsize=(12, 8))

# ML Types visualization
ml_types = ['Supervised', 'Unsupervised', 'Reinforcement']
examples = ['Classification\nRegression', 'Clustering\nDimensionality\nReinforcement']

colors = ['#FF6B6B', '#4ECDC4', '#45B7D1']
y_pos = [0.7, 0.5, 0.3]

for i, (ml_type, example, color) in enumerate(zip(ml_types, examples, colors)):
    ax.text(0.1, y_pos[i], f'{ml_type} Learning', fontsize=16, fontweight='bold')
    ax.text(0.1, y_pos[i]-0.05, example, fontsize=12, color=color)
    ax.barh(y_pos[i], 0.8, height=0.08, color=color, alpha=0.3)

ax.set_xlim(0, 1)
ax.set_ylim(0, 1)
ax.set_title('Machine Learning Types', fontsize=18, fontweight='bold', pad=10)
ax.axis('off')

plt.tight_layout()
plt.show()

print("\nKey Takeaway: Machine Learning is about teaching computers to learn from data")

```

2. Types of Machine Learning Problems

```

In [ ]: # =====
# 2. TYPES OF MACHINE LEARNING PROBLEMS
# =====

print("MACHINE LEARNING PROBLEM TYPES")
print("=" * 40)

# 1. CLASSIFICATION PROBLEMS
print("\n1. CLASSIFICATION")
print("    Goal: Predict categorical labels")
print("    Output: Discrete classes/categories")
print("    Examples:")
print("        • Binary: Spam/Not Spam, Sick/Healthy")
print("        • Multiclass: Image recognition (Cat/Dog/Bird)")
print("        • Multilabel: Multiple tags per image")

# Generate classification data example
X_class, y_class = make_classification(n_samples=200, n_features=2, n_redundant=0,
                                      n_informative=2, n_clusters_per_class=2,
                                      random_state=42)

# Visualize classification data
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

# Classification scatter plot
scatter = axes[0].scatter(X_class[:, 0], X_class[:, 1], c=y_class, cmap='viridis')
axes[0].set_title('Classification Problem\n(Two Classes)', fontsize=14, fontweight='bold')
axes[0].set_xlabel('Feature 1')
axes[0].set_ylabel('Feature 2')
plt.colorbar(scatter, ax=axes[0])

# 2. REGRESSION PROBLEMS
print("\n2. REGRESSION")
print("    Goal: Predict continuous numerical values")
print("    Output: Real numbers")
print("    Examples:")
print("        • House price prediction")
print("        • Stock price forecasting")
print("        • Temperature prediction")
print("        • Sales forecasting")

# Generate regression data example
X_reg, y_reg = make_regression(n_samples=200, n_features=1, noise=20, random_state=42)

# Regression scatter plot
axes[1].scatter(X_reg, y_reg, alpha=0.7, color='orange')
axes[1].set_title('Regression Problem\n(Continuous Output)', fontsize=14, fontweight='bold')
axes[1].set_xlabel('Feature')
axes[1].set_ylabel('Target Value')

plt.tight_layout()
plt.show()

# 3. CLUSTERING PROBLEMS
print("\n3. CLUSTERING (UNSUPERVISED)")
print("    Goal: Group similar data points")
print("    Output: Cluster assignments")
print("    Examples:")
print("        • Customer segmentation")

```

```

print("    • Gene sequencing")
print("    • Market research")
print("    • Anomaly detection")

# Generate clustering data
from sklearn.datasets import make_blobs
X_cluster, y_cluster = make_blobs(n_samples=200, centers=3, n_features=2,
                                  random_state=42, cluster_std=2)

# Visualize clustering data
plt.figure(figsize=(8, 6))
plt.scatter(X_cluster[:, 0], X_cluster[:, 1], c=y_cluster, cmap='Set1', a
plt.title('Clustering Problem\n(Group Similar Data Points)', fontsize=14,
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()

# 4. DIMENSIONALITY REDUCTION
print("\n4. DIMENSIONALITY REDUCTION")
print("    Goal: Reduce number of features while preserving information")
print("    Output: Lower-dimensional representation")
print("    Examples:")
print("    • Data visualization (2D/3D plots)")
print("    • Noise reduction")
print("    • Feature compression")
print("    • Data preprocessing")

# Demonstrate dimensionality reduction
from sklearn.decomposition import PCA
from sklearn.datasets import load_breast_cancer

# Load high-dimensional data
cancer = load_breast_cancer()
X_high_dim = cancer.data
y_high_dim = cancer.target

# Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_high_dim)

# Visualize dimensionality reduction
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y_high_dim, cmap='viridis', alpha
plt.title('Dimensionality Reduction\n(30D → 2D using PCA)', fontsize=14,
plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.1%} variance)')
plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.1%} variance)')

plt.subplot(1, 2, 2)
plt.bar(range(1, 11), pca.explained_variance_ratio_[1:10])
plt.title('Explained Variance by Component', fontsize=14, fontweight='bol
plt.xlabel('Principal Component')
plt.ylabel('Explained Variance Ratio')

plt.tight_layout()
plt.show()

print(f"\nPROBLEM TYPE SUMMARY")
print("=" * 25)

```

```

print(f"Classification: {len(np.unique(y_class))} classes, {X_class.shape[1]} features")
print(f"Regression: Continuous target, {X_reg.shape[1]} features")
print(f"Clustering: {len(np.unique(y_cluster))} clusters, {X_cluster.shape[1]} features")
print(f"Dimensionality Reduction: {X_high_dim.shape[1]}D → 2D")

print("\nKey Takeaway: Choose the right problem type based on your data a

```

3. Data Preprocessing and Feature Engineering

```

In [ ]: # =====
# 3. DATA PREPROCESSING AND FEATURE ENGINEERING
# =====

print("DATA PREPROCESSING & FEATURE ENGINEERING")
print("=" * 50)

print("\nGOALS:")
print("• Clean and prepare data for ML algorithms")
print("• Handle missing values, outliers, and inconsistencies")
print("• Transform categorical variables")
print("• Scale numerical features")
print("• Create new meaningful features")
print("• Select the most relevant features")

# Create a comprehensive dataset with various data types and issues
np.random.seed(42)
n_samples = 1000

print("\nCREATING SAMPLE DATASET")
print("-" * 30)

# Generate synthetic data with realistic patterns
data = {
    'age': np.random.normal(35, 10, n_samples),
    'income': np.random.normal(50000, 15000, n_samples),
    'education': np.random.choice(['High School', 'Bachelor', 'Master', 'PhD'], n_samples),
    'experience': np.random.randint(0, 30, n_samples),
    'city': np.random.choice(['New York', 'Los Angeles', 'Chicago', 'Houston', 'San Francisco'], n_samples),
    'department': np.random.choice(['IT', 'HR', 'Finance', 'Marketing', 'Operations'], n_samples),
    'performance_score': np.random.normal(75, 15, n_samples),
    'satisfaction': np.random.choice(['Low', 'Medium', 'High'], n_samples),
    'salary': np.random.normal(60000, 20000, n_samples),
    'years_at_company': np.random.randint(0, 15, n_samples)
}

# Create DataFrame
df = pd.DataFrame(data)

# Introduce realistic missing values (some patterns)
missing_indices = np.random.choice(df.index, size=int(0.1 * len(df)), replace=True)
df.loc[missing_indices, 'income'] = np.nan

missing_indices = np.random.choice(df.index, size=int(0.05 * len(df)), replace=True)
df.loc[missing_indices, 'education'] = np.nan

# Introduce outliers (high earners)
outlier_indices = np.random.choice(df.index, size=int(0.02 * len(df)), replace=True)
df.loc[outlier_indices, 'salary'] = df.loc[outlier_indices, 'salary'] * 3

```

```

print(f"Dataset created: {df.shape[0]} rows, {df.shape[1]} columns")
print(f"Features: {list(df.columns)}")

print("\nDATASET OVERVIEW")
print("-" * 20)
print(f"\nFirst 5 rows:")
print(df.head())

print(f"\nData types:")
print(df.dtypes)

print(f"\nMissing values:")
missing_summary = df.isnull().sum()
print(missing_summary[missing_summary > 0])

print(f"\nBasic statistics:")
print(df.describe())

# 1. HANDLING MISSING VALUES
print("\n1. HANDLING MISSING VALUES")
print("-" * 35)

# Check missing values
print(f"Missing values per column:")
missing_summary = df.isnull().sum()
print(missing_summary[missing_summary > 0])

# Visualize missing values
plt.figure(figsize=(10, 6))
plt.subplot(1, 2, 1)
missing_summary[missing_summary > 0].plot(kind='bar', color='coral')
plt.title('Missing Values by Column', fontweight='bold')
plt.ylabel('Count')
plt.xticks(rotation=45)

# Strategy 1: Drop rows with missing values
df_dropna = df.dropna()
print(f"\nStrategy 1 – Drop missing values:")
print(f"    Original: {df.shape[0]} rows")
print(f"    After drop: {df_dropna.shape[0]} rows")
print(f"    Lost: {df.shape[0] - df_dropna.shape[0]} rows ({((df.shape[0] - df_dropna.shape[0]) / df.shape[0]) * 100}%)")

# Strategy 2: Fill missing values with mean/median/mode
df_fillna = df.copy()
df_fillna['income'] = df_fillna['income'].fillna(df_fillna['income'].median())
df_fillna['education'] = df_fillna['education'].fillna(df_fillna['education'].mode()[0])
print(f"\nStrategy 2 – Fill missing values:")
print(f"    Income: filled with median ({df['income'].median():.0f})")
print(f"    Education: filled with mode ({df['education'].mode()[0]})")

# Strategy 3: Use SimpleImputer
imputer = SimpleImputer(strategy='median')
df_imputed = df.copy()
df_imputed['income'] = imputer.fit_transform(df_imputed[['income']]).flat
print(f"\nStrategy 3 – SimpleImputer:")
print(f"    Used median strategy for numerical features")

# Compare strategies
plt.subplot(1, 2, 2)

```

```

strategies = ['Original', 'Drop NA', 'Fill NA', 'Imputer']
missing_counts = [df.isnull().sum().sum(), df_dropna.isnull().sum().sum(),
                  df_fillna.isnull().sum().sum(), df_imputed.isnull().sum().sum()]
plt.bar(strategies, missing_counts, color=['red', 'orange', 'green', 'blue'])
plt.title('Missing Values After Each Strategy', fontweight='bold')
plt.ylabel('Total Missing Values')
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()

print(f"\nMissing values handled successfully!")

# 2. Handling Categorical Variables
print("\n=== HANDLING CATEGORICAL VARIABLES ===")

# Label Encoding
le = LabelEncoder()
df_encoded = df_fillna.copy()
df_encoded['education_encoded'] = le.fit_transform(df_encoded['education'])
df_encoded['city_encoded'] = le.fit_transform(df_encoded['city'])
df_encoded['department_encoded'] = le.fit_transform(df_encoded['department'])
df_encoded['satisfaction_encoded'] = le.fit_transform(df_encoded['satisfaction'])

print(f"\nLabel encoded values:")
print(f"Education: {dict(zip(le.classes_, le.transform(le.classes_)))}")
print(f"City: {dict(zip(le.classes_, le.transform(le.classes_)))}")

# One-Hot Encoding
df_onehot = df_fillna.copy()
education_dummies = pd.get_dummies(df_onehot['education'], prefix='education')
city_dummies = pd.get_dummies(df_onehot['city'], prefix='city')
department_dummies = pd.get_dummies(df_onehot['department'], prefix='department')
satisfaction_dummies = pd.get_dummies(df_onehot['satisfaction'], prefix='satisfaction')

df_onehot = pd.concat([df_onehot, education_dummies, city_dummies, department_dummies, satisfaction_dummies])
print(f"\nAfter one-hot encoding: {df_onehot.shape}")

# 3. Feature Scaling
print("\n=== FEATURE SCALING ===")

# Select numerical features for scaling
numerical_features = ['age', 'income', 'experience', 'performance_score', 'X_numerical']
X_numerical = df_fillna[numerical_features]

# Standardization (mean=0, std=1)
scaler_standard = StandardScaler()
X_standardized = scaler_standard.fit_transform(X_numerical)
print(f"\nStandardized features shape: {X_standardized.shape}")
print(f"Mean: {X_standardized.mean(axis=0)}")
print(f"Std: {X_standardized.std(axis=0)}")

# Min-Max Scaling (0-1 range)
scaler_minmax = MinMaxScaler()
X_minmax = scaler_minmax.fit_transform(X_numerical)
print(f"\nMin-Max scaled features shape: {X_minmax.shape}")
print(f"Min: {X_minmax.min(axis=0)}")
print(f"Max: {X_minmax.max(axis=0)}")

# 4. Feature Engineering

```



```

print("\n=== FEATURE ENGINEERING ===")

# Create new features
df_features = df_fillna.copy()

# Age groups
df_features['age_group'] = pd.cut(df_features['age'],
                                  bins=[0, 25, 35, 45, 100],
                                  labels=['Young', 'Middle', 'Senior', 'El

# Income categories
df_features['income_category'] = pd.cut(df_features['income'],
                                         bins=[0, 30000, 50000, 70000, 1000
                                         labels=['Low', 'Medium', 'High', '

# Experience level
df_features['experience_level'] = pd.cut(df_features['experience'],
                                         bins=[0, 5, 10, 20, 100],
                                         labels=['Junior', 'Mid', 'Senior'

# Performance categories
df_features['performance_category'] = pd.cut(df_features['performance_sco
                                             bins=[0, 60, 80, 90, 100],
                                             labels=['Poor', 'Good', 'Very

# Salary ratio
df_features['salary_ratio'] = df_features['salary'] / df_features['income

# Experience to age ratio
df_features['experience_age_ratio'] = df_features['experience'] / df_feat

print(f"\nNew features created:")
print(df_features[['age_group', 'income_category', 'experience_level', 'p

# 5. Feature Selection
print("\n=== FEATURE SELECTION ===")

# Prepare features and target
X = df_features[['age', 'income', 'experience', 'performance_score']]
y = df_features['satisfaction']

# Select top 2 features
selector = SelectKBest(score_func=f_classif, k=2)
X_selected = selector.fit_transform(X, y)

print(f"\nSelected features: {X.columns[selector.get_support()].tolist()}")
print(f"Feature scores: {selector.scores_}")
print(f"Selected features shape: {X_selected.shape}")

# 6. Data Splitting
print("\n=== DATA SPLITTING ===")

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

print(f"Training set size: {X_train.shape[0]}")
print(f"Test set size: {X_test.shape[0]}")
print(f"Training set target distribution:")
print(y_train.value_counts(normalize=True))
print(f"\nTest set target distribution:")

```

```

print(y_test.value_counts(normalize=True))

# 7. Complete Preprocessing Pipeline
print("\n=== COMPLETE PREPROCESSING PIPELINE ===")

# Create a complete preprocessing pipeline
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

# Define numerical and categorical features
numerical_features = ['age', 'income', 'experience', 'performance_score']
categorical_features = ['education', 'city', 'department']

# Create preprocessing pipelines
numerical_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

# Combine pipelines
preprocessor = ColumnTransformer([
    ('num', numerical_pipeline, numerical_features),
    ('cat', categorical_pipeline, categorical_features)
])

# Apply preprocessing
X_processed = preprocessor.fit_transform(df_fillna)
print(f"\nProcessed features shape: {X_processed.shape}")
print(f"Feature names: {preprocessor.get_feature_names_out()}")

print("\n=== PREPROCESSING SUMMARY ===")
print("1. Missing values handled with appropriate strategies")
print("2. Categorical variables encoded (label and one-hot)")
print("3. Features scaled (standardization and min-max)")
print("4. New features created through engineering")
print("5. Feature selection applied")
print("6. Data split into train/test sets")
print("7. Complete pipeline created for reproducibility")

```

```

In [ ]: # 2. HANDLING CATEGORICAL VARIABLES
print("\n2. HANDLING CATEGORICAL VARIABLES")
print("-" * 40)

# Work with the filled dataset
df_work = df_fillna.copy()

print("Categorical columns:")
categorical_cols = df_work.select_dtypes(include=['object']).columns.tolist()
print(f"    {categorical_cols}")

# Label Encoding
print("\nLabel Encoding:")
le = LabelEncoder()
df_encoded = df_work.copy()

```

```

for col in ['education', 'city', 'department', 'satisfaction']:
    df_encoded[f'{col}_encoded'] = le.fit_transform(df_encoded[col])
    print(f"    {col}: {dict(zip(le.classes_, le.transform(le.classes_)))}"

# One-Hot Encoding
print("\n\nOne-Hot Encoding:")
df_onehot = df_work.copy()

# Create dummy variables
education_dummies = pd.get_dummies(df_onehot['education'], prefix='educat
city_dummies = pd.get_dummies(df_onehot['city'], prefix='city')
department_dummies = pd.get_dummies(df_onehot['department'], prefix='depa
satisfaction_dummies = pd.get_dummies(df_onehot['satisfaction'], prefix='

# Combine with original data
df_onehot = pd.concat([df_onehot, education_dummies, city_dummies,
                      department_dummies, satisfaction_dummies], axis=1)

print(f"    Original features: {df_work.shape[1]}")
print(f"    After one-hot: {df_onehot.shape[1]}")
print(f"    Added: {df_onehot.shape[1] - df_work.shape[1]} new features")

# Visualize encoding comparison
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Original categorical distribution
df_work['education'].value_counts().plot(kind='bar', ax=axes[0], color='s
axes[0].set_title('Original Education Distribution', fontweight='bold')
axes[0].set_ylabel('Count')
axes[0].tick_params(axis='x', rotation=45)

# Label encoded
df_encoded['education_encoded'].value_counts().plot(kind='bar', ax=axes[1]
axes[1].set_title('Label Encoded Education', fontweight='bold')
axes[1].set_ylabel('Count')
axes[1].set_xlabel('Encoded Value')

# One-hot encoded (show first few)
education_cols = [col for col in df_onehot.columns if col.startswith('edu
df_onehot[education_cols].sum().plot(kind='bar', ax=axes[2], color='coral
axes[2].set_title('One-Hot Encoded Education', fontweight='bold')
axes[2].set_ylabel('Count')
axes[2].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()

print("\n\nCategorical variables encoded successfully!")

```

```

In [ ]: # 3. FEATURE SCALING
print("\n\n3. FEATURE SCALING")
print("-" * 25)

# Select numerical features for scaling
numerical_features = ['age', 'income', 'experience', 'performance_score',
X_numerical = df_work[numerical_features]

print(f"Numerical features to scale: {numerical_features}")
print(f"\n\nOriginal statistics:")
print(X_numerical.describe())

```



```

# Performance categories
df_features['performance_category'] = pd.cut(df_features['performance_score'],
                                              bins=[0, 60, 80, 90, 100],
                                              labels=['Poor', 'Good', 'Very Good'])

# Mathematical combinations
df_features['salary_ratio'] = df_features['salary'] / df_features['income']
df_features['experience_age_ratio'] = df_features['experience'] / df_features['age']
df_features['performance_income_ratio'] = df_features['performance_score'] / df_features['income']

# Interaction features
df_features['age_experience_interaction'] = df_features['age'] * df_features['experience']
df_features['income_performance_interaction'] = df_features['income'] * df_features['performance_score']

# Polynomial features
df_features['age_squared'] = df_features['age'] ** 2
df_features['income_squared'] = df_features['income'] ** 2

print(f"\nCreated {len(df_features.columns) - len(df_work.columns)} new features")
print("\nNew features:")
new_features = [col for col in df_features.columns if col not in df_work.columns]
for i, feature in enumerate(new_features, 1):
    print(f"    {i:2d}. {feature}")

# Visualize some new features
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.flatten()

# Age groups distribution
df_features['age_group'].value_counts().plot(kind='bar', ax=axes[0], color='blue')
axes[0].set_title('Age Groups Distribution', fontweight='bold')
axes[0].tick_params(axis='x', rotation=45)

# Income categories
df_features['income_category'].value_counts().plot(kind='bar', ax=axes[1], color='green')
axes[1].set_title('Income Categories', fontweight='bold')
axes[1].tick_params(axis='x', rotation=45)

# Experience levels
df_features['experience_level'].value_counts().plot(kind='bar', ax=axes[2], color='red')
axes[2].set_title('Experience Levels', fontweight='bold')
axes[2].tick_params(axis='x', rotation=45)

# Salary ratio distribution
axes[3].hist(df_features['salary_ratio'], bins=30, color='purple', alpha=0.5)
axes[3].set_title('Salary Ratio Distribution', fontweight='bold')
axes[3].set_xlabel('Salary/Income Ratio')

# Experience-Age ratio
axes[4].hist(df_features['experience_age_ratio'], bins=30, color='orange', alpha=0.5)
axes[4].set_title('Experience/Age Ratio', fontweight='bold')
axes[4].set_xlabel('Experience/Age')

# Performance categories
df_features['performance_category'].value_counts().plot(kind='bar', ax=axes[5], color='brown')
axes[5].set_title('Performance Categories', fontweight='bold')
axes[5].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()

```

```
print("\nFeature engineering completed successfully!")
```

4. Train-Test Split and Cross-Validation

```
In [ ]: # =====
# 4. TRAIN-TEST SPLIT AND CROSS-VALIDATION
# =====

print("TRAIN-TEST SPLIT AND CROSS-VALIDATION")
print("=" * 45)

print("\nWHY SPLIT DATA?")
print("• Prevent overfitting")
print("• Get unbiased model performance estimate")
print("• Simulate real-world performance")
print("• Compare different models fairly")

# Prepare data for splitting
numerical_features = ['age', 'income', 'experience', 'performance_score',
X = df_features[numerical_features]
y = df_features['satisfaction']

print(f"\nDataset for splitting:")
print(f"  Features shape: {X.shape}")
print(f"  Target shape: {y.shape}")
print(f"  Target distribution:")
print(y.value_counts(normalize=True).round(3))

# 1. BASIC TRAIN-TEST SPLIT
print("\n1. BASIC TRAIN-TEST SPLIT")
print("-" * 35)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

print(f"  Training set: {X_train.shape[0]} samples ({X_train.shape[0]/le
print(f"  Test set: {X_test.shape[0]} samples ({X_test.shape[0]/len(X)*1

print(f"\n  Training target distribution:")
print(y_train.value_counts(normalize=True).round(3))
print(f"\n  Test target distribution:")
print(y_test.value_counts(normalize=True).round(3))

# 2. CROSS-VALIDATION
print("\n2. CROSS-VALIDATION")
print("-" * 25)

# K-Fold Cross-Validation
print("\nK-Fold Cross-Validation:")
kfold = KFold(n_splits=5, shuffle=True, random_state=42)

# Create a simple model for demonstration
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(random_state=42)

# Perform cross-validation
cv_scores = cross_val_score(model, X_train, y_train, cv=kfold, scoring='a
```

```

print(f"    CV Scores: {cv_scores.round(3)}")
print(f"    Mean CV Score: {cv_scores.mean():.3f} (+/- {cv_scores.std() *

# Stratified K-Fold (better for imbalanced datasets)
print("\nStratified K-Fold Cross-Validation:")
skfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_scores_stratified = cross_val_score(model, X_train, y_train, cv=skfold

print(f"    Stratified CV Scores: {cv_scores_stratified.round(3)}")
print(f"    Mean Stratified CV Score: {cv_scores_stratified.mean():.3f} (+

# Visualize cross-validation results
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

# K-Fold scores
axes[0].bar(range(1, 6), cv_scores, color='skyblue', alpha=0.7)
axes[0].axhline(y=cv_scores.mean(), color='red', linestyle='--', label=f'
axes[0].set_title('K-Fold Cross-Validation Scores', fontweight='bold')
axes[0].set_xlabel('Fold')
axes[0].set_ylabel('Accuracy')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Stratified K-Fold scores
axes[1].bar(range(1, 6), cv_scores_stratified, color='lightgreen', alpha=
axes[1].axhline(y=cv_scores_stratified.mean(), color='red', linestyle='--
                label=f'Mean: {cv_scores_stratified.mean():.3f}')
axes[1].set_title('Stratified K-Fold Cross-Validation Scores', fontweight
axes[1].set_xlabel('Fold')
axes[1].set_ylabel('Accuracy')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# 3. VALIDATION SET (TRAIN-VALIDATION-TEST)
print("\n3. TRAIN-VALIDATION-TEST SPLIT")
print("-" * 40)

# First split: train+val vs test
X_temp, X_test_final, y_temp, y_test_final = train_test_split(X, y, test_

# Second split: train vs validation
X_train_final, X_val, y_train_final, y_val = train_test_split(X_temp, y_t

print(f"    Training set: {X_train_final.shape[0]} samples")
print(f"    Validation set: {X_val.shape[0]} samples")
print(f"    Test set: {X_test_final.shape[0]} samples")

print(f"\n    Training: {X_train_final.shape[0]/len(X)*100:.1f}%")
print(f"    Validation: {X_val.shape[0]/len(X)*100:.1f}%")
print(f"    Test: {X_test_final.shape[0]/len(X)*100:.1f}%")

# 4. TIME SERIES SPLIT (for temporal data)
print("\n4. TIME SERIES SPLIT")
print("-" * 25)

from sklearn.model_selection import TimeSeriesSplit

```

```
# Create time series data
np.random.seed(42)
time_data = np.random.randn(100, 3)
time_target = np.random.randn(100)

tscv = TimeSeriesSplit(n_splits=5)
print("    Time series splits:")
for i, (train_idx, val_idx) in enumerate(tscv.split(time_data)):
    print(f"    Split {i+1}: Train {len(train_idx)} samples, Validation {len(val_idx)} samples")

print("\nData splitting strategies completed!")
print("\nKey Takeaways:")
print("• Always split data before any preprocessing")
print("• Use stratified split for imbalanced datasets")
print("• Cross-validation gives more robust performance estimates")
print("• Keep test set completely separate until final evaluation")
```

5. Introduction to scikit-learn

```
In [ ]: # =====
# 5. INTRODUCTION TO SCIKIT-LEARN
# =====

print("INTRODUCTION TO SCIKIT-LEARN")
print("=" * 40)

print("\nWHAT IS SCIKIT-LEARN?")
print("• Most popular Python ML library")
print("• Built on NumPy, SciPy, and Matplotlib")
print("• Consistent API across all algorithms")
print("• Extensive documentation and community")
print("• Production-ready and well-tested")

print("\nKEY COMPONENTS:")
print("• sklearn.datasets: Sample datasets")
print("• sklearn.preprocessing: Data preprocessing")
print("• sklearn.model_selection: Model selection and validation")
print("• sklearn.linear_model: Linear models")
print("• sklearn.tree: Decision trees")
print("• sklearn.ensemble: Ensemble methods")
print("• sklearn.metrics: Model evaluation")
print("• sklearn.pipeline: Pipeline construction")

# 1. DATASETS
print("\n1. WORKING WITH DATASETS")
print("-" * 35)

# Load built-in datasets
from sklearn.datasets import load_iris, load_wine, load_breast_cancer

# Load iris dataset
iris = load_iris()
print(f"\nIris Dataset:")
print(f"    Features: {iris.data.shape[1]}")
print(f"    Samples: {iris.data.shape[0]}")
print(f"    Classes: {len(iris.target_names)}")
print(f"    Target names: {iris.target_names}")
```



```

# Load wine dataset
wine = load_wine()
print(f"\nWine Dataset:")
print(f"    Features: {wine.data.shape[1]}")
print(f"    Samples: {wine.data.shape[0]}")
print(f"    Classes: {len(wine.target_names)}")

# Create synthetic datasets
print(f"\nSynthetic Datasets:")
X_class, y_class = make_classification(n_samples=100, n_features=4, n_classes=3, random_state=42)
X_reg, y_reg = make_regression(n_samples=100, n_features=3, noise=10, random_state=42)

print(f"    Classification: {X_class.shape[0]} samples, {X_class.shape[1]} features")
print(f"    Regression: {X_reg.shape[0]} samples, {X_reg.shape[1]} features")

# 2. CONSISTENT API
print("\n2. CONSISTENT API PATTERN")
print("-" * 35)

print("\nStandard ML Workflow:")
print("1. Create model instance")
print("2. Fit model to training data")
print("3. Make predictions on test data")
print("4. Evaluate model performance")

# Example with different algorithms
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

print("\nExample with different algorithms:")

# Linear Regression
print("\nLinear Regression:")
lr = LinearRegression()
lr.fit(X_reg, y_reg)
lr_pred = lr.predict(X_reg[:5])
print(f"    Predictions: {lr_pred.round(2)}")

# Decision Tree
print("\nDecision Tree:")
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_class, y_class)
dt_pred = dt.predict(X_class[:5])
print(f"    Predictions: {dt_pred}")

# Random Forest
print("\nRandom Forest:")
rf = RandomForestClassifier(random_state=42)
rf.fit(X_class, y_class)
rf_pred = rf.predict(X_class[:5])
print(f"    Predictions: {rf_pred}")

# 3. PIPELINES
print("\n3. PIPELINES")
print("-" * 20)

print("\nPipeline Benefits:")
print("• Chain multiple preprocessing steps")
print("• Prevent data leakage")

```

```

print("• Make code more readable")
print("• Easy to deploy and reproduce")

# Create a preprocessing pipeline
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

# Simple pipeline
pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', LogisticRegression(random_state=42))
])

print("\nPipeline created:")
print(pipe)

# Fit and predict with pipeline
pipe.fit(X_class, y_class)
pipe_pred = pipe.predict(X_class[:5])
print(f"\n Pipeline predictions: {pipe_pred}")

# 4. MODEL PERSISTENCE
print("\n4. MODEL PERSISTENCE")
print("-" * 30)

import joblib

# Save model
joblib.dump(pipe, 'ml_pipeline.pkl')
print(" Model saved to 'ml_pipeline.pkl'")

# Load model
loaded_pipe = joblib.load('ml_pipeline.pkl')
print(" Model loaded from file")

# Verify loaded model works
loaded_pred = loaded_pipe.predict(X_class[:5])
print(f" Loaded model predictions: {loaded_pred}")

# 5. VISUALIZATION
print("\n5. SCIKIT-LEARN VISUALIZATION")
print("-" * 40)

# Plot decision boundary for 2D data
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.inspection import DecisionBoundaryDisplay

# Create 2D classification data
X_2d, y_2d = make_classification(n_samples=200, n_features=2, n_redundant
                                n_informative=2, n_clusters_per_class=1,

# Train logistic regression
lr_2d = LogisticRegression(random_state=42)
lr_2d.fit(X_2d, y_2d)

# Plot decision boundary
fig, ax = plt.subplots(1, 1, figsize=(10, 8))
DecisionBoundaryDisplay.from_estimator(lr_2d, X_2d, ax=ax, alpha=0.3)
ax.scatter(X_2d[:, 0], X_2d[:, 1], c=y_2d, cmap='viridis', edgecolors='bl

```

```

ax.set_title('Logistic Regression Decision Boundary', fontsize=14, fontwe
ax.set_xlabel('Feature 1')
ax.set_ylabel('Feature 2')
plt.show()

print("\nScikit-learn introduction completed!")
print("\nKey Takeaways:")
print("• Consistent API makes learning easier")
print("• Pipelines prevent data leakage")
print("• Built-in datasets for practice")
print("• Excellent documentation and community support")
print("• Production-ready and well-tested")

```

6. Linear Regression

```

In [ ]: # =====
# 6. LINEAR REGRESSION
# =====

print("LINEAR REGRESSION")
print("=" * 25)

print("\nWHAT IS LINEAR REGRESSION?")
print("• Predicts continuous numerical values")
print("• Finds best linear relationship between features and target")
print("• Assumes linear relationship:  $y = \beta_0 + \beta_1x_1 + \beta_2x_2 + \dots + \beta_nx_n$ ")
print("• Minimizes sum of squared errors")

print("\nWHEN TO USE:")
print("• Target variable is continuous")
print("• Linear relationship exists")
print("• Features are independent")
print("• No multicollinearity")

# 1. SIMPLE LINEAR REGRESSION
print("\n1. SIMPLE LINEAR REGRESSION")
print("-" * 40)

# Generate sample data
np.random.seed(42)
X_simple = np.random.randn(100, 1) * 10
y_simple = 2 * X_simple.flatten() + 3 + np.random.randn(100) * 2

# Split data
X_train_simple, X_test_simple, y_train_simple, y_test_simple = train_test
    X_simple, y_simple, test_size=0.2, random_state=42)

# Create and train model
lr_simple = LinearRegression()
lr_simple.fit(X_train_simple, y_train_simple)

# Make predictions
y_pred_simple = lr_simple.predict(X_test_simple)

# Calculate metrics
mse_simple = mean_squared_error(y_test_simple, y_pred_simple)
r2_simple = r2_score(y_test_simple, y_pred_simple)
mae_simple = mean_absolute_error(y_test_simple, y_pred_simple)

```

```

print(f"\nResults:")
print(f"    Coefficient ( $\beta_1$ ): {lr_simple.coef_[0]:.3f}")
print(f"    Intercept ( $\beta_0$ ): {lr_simple.intercept_: .3f}")
print(f"    Mean Squared Error: {mse_simple:.3f}")
print(f"    R-squared: {r2_simple:.3f}")
print(f"    Mean Absolute Error: {mae_simple:.3f}")

# Visualize simple linear regression
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

# Training data
axes[0].scatter(X_train_simple, y_train_simple, color='blue', alpha=0.6, label='Training')
axes[0].plot(X_train_simple, lr_simple.predict(X_train_simple), color='red', linewidth=2)
axes[0].set_title('Simple Linear Regression - Training', fontweight='bold')
axes[0].set_xlabel('X')
axes[0].set_ylabel('y')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Test data
axes[1].scatter(X_test_simple, y_test_simple, color='blue', alpha=0.6, label='Test')
axes[1].scatter(X_test_simple, y_pred_simple, color='red', alpha=0.6, label='Predictions')
axes[1].plot(X_test_simple, y_pred_simple, color='red', linewidth=2)
axes[1].set_title('Simple Linear Regression - Testing', fontweight='bold')
axes[1].set_xlabel('X')
axes[1].set_ylabel('y')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# 2. MULTIPLE LINEAR REGRESSION
print("\n📊 2. MULTIPLE LINEAR REGRESSION")
print("-" * 40)

# Generate multiple features
X_multi, y_multi = make_regression(n_samples=200, n_features=3, noise=20, random_state=42)

# Split data
X_train_multi, X_test_multi, y_train_multi, y_test_multi = train_test_split(X_multi, y_multi, test_size=0.2, random_state=42)

# Create and train model
lr_multi = LinearRegression()
lr_multi.fit(X_train_multi, y_train_multi)

# Make predictions
y_pred_multi = lr_multi.predict(X_test_multi)

# Calculate metrics
mse_multi = mean_squared_error(y_test_multi, y_pred_multi)
r2_multi = r2_score(y_test_multi, y_pred_multi)
mae_multi = mean_absolute_error(y_test_multi, y_pred_multi)

print(f"\n📊 Results:")
print(f"    Coefficients: {lr_multi.coef_}")
print(f"    Intercept: {lr_multi.intercept_: .3f}")
print(f"    Mean Squared Error: {mse_multi:.3f}")

```

```

print(f"    R-squared: {r2_multi:.3f}")
print(f"    Mean Absolute Error: {mae_multi:.3f}")

# Feature importance (coefficient magnitude)
feature_names = [f'Feature {i+1}' for i in range(X_multi.shape[1])]
coef_df = pd.DataFrame({
    'Feature': feature_names,
    'Coefficient': lr_multi.coef_,
    'Abs_Coefficient': np.abs(lr_multi.coef_)
}).sort_values('Abs_Coefficient', ascending=False)

# Visualize feature importance
plt.figure(figsize=(10, 6))
plt.barh(coef_df['Feature'], coef_df['Coefficient'], color='skyblue')
plt.title('Feature Importance (Coefficients)', fontweight='bold')
plt.xlabel('Coefficient Value')
plt.grid(True, alpha=0.3)
plt.show()

# 3. RESIDUAL ANALYSIS
print("\n 3. RESIDUAL ANALYSIS")
print("-" * 30)

# Calculate residuals
residuals = y_test_multi - y_pred_multi

# Residual plots
fig, axes = plt.subplots(2, 2, figsize=(15, 10))

# Residuals vs Predicted
axes[0, 0].scatter(y_pred_multi, residuals, alpha=0.6)
axes[0, 0].axhline(y=0, color='red', linestyle='--')
axes[0, 0].set_title('Residuals vs Predicted', fontweight='bold')
axes[0, 0].set_xlabel('Predicted Values')
axes[0, 0].set_ylabel('Residuals')
axes[0, 0].grid(True, alpha=0.3)

# Residuals histogram
axes[0, 1].hist(residuals, bins=20, alpha=0.7, color='skyblue')
axes[0, 1].set_title('Residuals Distribution', fontweight='bold')
axes[0, 1].set_xlabel('Residuals')
axes[0, 1].set_ylabel('Frequency')
axes[0, 1].grid(True, alpha=0.3)

# Q-Q plot for normality
from scipy import stats
stats.probplot(residuals, dist="norm", plot=axes[1, 0])
axes[1, 0].set_title('Q-Q Plot (Normality Check)', fontweight='bold')
axes[1, 0].grid(True, alpha=0.3)

# Actual vs Predicted
axes[1, 1].scatter(y_test_multi, y_pred_multi, alpha=0.6)
axes[1, 1].plot([y_test_multi.min(), y_test_multi.max()],
                [y_test_multi.min(), y_test_multi.max()], 'r--', lw=2)
axes[1, 1].set_title('Actual vs Predicted', fontweight='bold')
axes[1, 1].set_xlabel('Actual Values')
axes[1, 1].set_ylabel('Predicted Values')
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()

```

```

plt.show()

# 4. POLYNOMIAL REGRESSION
print("\n✅ 4. POLYNOMIAL REGRESSION")
print("-" * 35)

from sklearn.preprocessing import PolynomialFeatures

# Generate non-linear data
X_poly = np.linspace(-3, 3, 100).reshape(-1, 1)
y_poly = 0.5 * X_poly.flatten()**3 - 2 * X_poly.flatten()**2 + X_poly.fla

# Create polynomial features
poly_features = PolynomialFeatures(degree=3)
X_poly_features = poly_features.fit_transform(X_poly)

# Train polynomial regression
lr_poly = LinearRegression()
lr_poly.fit(X_poly_features, y_poly)

# Make predictions
y_pred_poly = lr_poly.predict(X_poly_features)

# Visualize polynomial regression
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.scatter(X_poly, y_poly, alpha=0.6, color='blue', label='Data')
plt.plot(X_poly, y_pred_poly, color='red', linewidth=2, label='Polynomial')
plt.title('Polynomial Regression (Degree 3)', fontweight='bold')
plt.xlabel('X')
plt.ylabel('y')
plt.legend()
plt.grid(True, alpha=0.3)

# Compare different degrees
plt.subplot(1, 2, 2)
degrees = [1, 2, 3, 4]
colors = ['blue', 'green', 'red', 'purple']

for degree, color in zip(degrees, colors):
    poly_features_deg = PolynomialFeatures(degree=degree)
    X_poly_deg = poly_features_deg.fit_transform(X_poly)
    lr_deg = LinearRegression()
    lr_deg.fit(X_poly_deg, y_poly)
    y_pred_deg = lr_deg.predict(X_poly_deg)
    plt.plot(X_poly, y_pred_deg, color=color, linewidth=2, label=f'Degree {degree}')

plt.scatter(X_poly, y_poly, alpha=0.3, color='black', label='Data')
plt.title('Polynomial Regression Comparison', fontweight='bold')
plt.xlabel('X')
plt.ylabel('y')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\n✅ Linear Regression completed!")
print("\n💡 Key Takeaways:")

```

```

print("• Linear regression finds the best linear relationship")
print("• Residual analysis helps validate assumptions")
print("• Polynomial regression can capture non-linear patterns")
print("• R2 measures how well the model explains variance")
print("• Feature importance can be assessed through coefficients")

```

7. Logistic Regression

```

In [ ]: # =====
# 7. LOGISTIC REGRESSION
# =====

print("\n📊 LOGISTIC REGRESSION")
print("=" * 30)

print("\n💡 WHAT IS LOGISTIC REGRESSION?")
print("• Classification algorithm (not regression despite the name)")
print("• Uses logistic function to model probability")
print("• Outputs probabilities between 0 and 1")
print("• Uses sigmoid function:  $p = 1 / (1 + e^{(-z)})$ ")
print("• Where  $z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$ ")

print("\n📌 WHEN TO USE:")
print("• Binary classification problems")
print("• Need probability estimates")
print("• Linear decision boundary is appropriate")
print("• Interpretable coefficients are important")

# 1. BINARY LOGISTIC REGRESSION
print("\n📋 1. BINARY LOGISTIC REGRESSION")
print("-" * 40)

# Generate binary classification data
X_binary, y_binary = make_classification(n_samples=200, n_features=2, n_r
                                       n_informative=2, n_clusters_per_c
                                       random_state=42)

# Split data
X_train_binary, X_test_binary, y_train_binary, y_test_binary = train_test
    X_binary, y_binary, test_size=0.2, random_state=42, stratify=y_binary

# Create and train logistic regression
lr_binary = LogisticRegression(random_state=42)
lr_binary.fit(X_train_binary, y_train_binary)

# Make predictions
y_pred_binary = lr_binary.predict(X_test_binary)
y_pred_proba_binary = lr_binary.predict_proba(X_test_binary)

# Calculate metrics
accuracy_binary = accuracy_score(y_test_binary, y_pred_binary)
precision_binary = precision_score(y_test_binary, y_pred_binary)
recall_binary = recall_score(y_test_binary, y_pred_binary)
f1_binary = f1_score(y_test_binary, y_pred_binary)

print(f"\n📊 Results:")
print(f"    Accuracy: {accuracy_binary:.3f}")
print(f"    Precision: {precision_binary:.3f}")

```

```

print(f"    Recall: {recall_binary:.3f}")
print(f"    F1-Score: {f1_binary:.3f}")
print(f"    Coefficients: {lr_binary.coef_[0]}")
print(f"    Intercept: {lr_binary.intercept_[0]:.3f}")

# Visualize decision boundary
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

# Decision boundary
DecisionBoundaryDisplay.from_estimator(lr_binary, X_binary, ax=axes[0], a
axes[0].scatter(X_binary[:, 0], X_binary[:, 1], c=y_binary, cmap='viridis'
axes[0].set_title('Logistic Regression Decision Boundary', fontweight='bo
axes[0].set_xlabel('Feature 1')
axes[0].set_ylabel('Feature 2')

# Probability predictions
scatter = axes[1].scatter(X_test_binary[:, 0], X_test_binary[:, 1],
                        c=y_pred_proba_binary[:, 1], cmap='coolwarm',
                        edgecolors='black', s=100)
axes[1].set_title('Predicted Probabilities', fontweight='bold')
axes[1].set_xlabel('Feature 1')
axes[1].set_ylabel('Feature 2')
plt.colorbar(scatter, ax=axes[1], label='Probability of Class 1')

plt.tight_layout()
plt.show()

# 2. MULTICLASS LOGISTIC REGRESSION
print("\n📦 2. MULTICLASS LOGISTIC REGRESSION")
print("-" * 45)

# Generate multiclass data
X_multi, y_multi = make_classification(n_samples=300, n_features=2, n_red
                                     n_informative=2, n_classes=3, n_clu
                                     random_state=42)

# Split data
X_train_multi, X_test_multi, y_train_multi, y_test_multi = train_test_spl
    X_multi, y_multi, test_size=0.2, random_state=42, stratify=y_multi)

# Create and train multiclass logistic regression
lr_multi = LogisticRegression(random_state=42, multi_class='ovr') # One-
lr_multi.fit(X_train_multi, y_train_multi)

# Make predictions
y_pred_multi = lr_multi.predict(X_test_multi)
y_pred_proba_multi = lr_multi.predict_proba(X_test_multi)

# Calculate metrics
accuracy_multi = accuracy_score(y_test_multi, y_pred_multi)

print(f"\n📊 Results:")
print(f"    Accuracy: {accuracy_multi:.3f}")
print(f"    Number of classes: {len(np.unique(y_multi))}")
print(f"    Coefficients shape: {lr_multi.coef_.shape}")

# Classification report
print(f"\n📋 Classification Report:")
print(classification_report(y_test_multi, y_pred_multi))

```



```

# Visualize multiclass decision boundaries
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

# Decision boundaries
DecisionBoundaryDisplay.from_estimator(lr_multi, X_multi, ax=axes[0], alp
axes[0].scatter(X_multi[:, 0], X_multi[:, 1], c=y_multi, cmap='viridis',
axes[0].set_title('Multiclass Decision Boundaries', fontweight='bold')
axes[0].set_xlabel('Feature 1')
axes[0].set_ylabel('Feature 2')

# Confusion matrix
cm = confusion_matrix(y_test_multi, y_pred_multi)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[1])
axes[1].set_title('Confusion Matrix', fontweight='bold')
axes[1].set_xlabel('Predicted')
axes[1].set_ylabel('Actual')

plt.tight_layout()
plt.show()

# 3. ROC CURVE AND AUC
print("\n📌 3. ROC CURVE AND AUC")
print("-" * 30)

# Calculate ROC curve for binary classification
fpr, tpr, thresholds = roc_curve(y_test_binary, y_pred_proba_binary[:, 1])
auc_score = roc_auc_score(y_test_binary, y_pred_proba_binary[:, 1])

# Plot ROC curve
plt.figure(figsize=(10, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC = {au
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Rando
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve', fontweight='bo
plt.legend(loc="lower right")
plt.grid(True, alpha=0.3)
plt.show()

print(f"    AUC Score: {auc_score:.3f}")

# 4. PROBABILITY CALIBRATION
print("\n🎯 4. PROBABILITY CALIBRATION")
print("-" * 35)

from sklearn.calibration import CalibratedClassifierCV, calibration_curve

# Create uncalibrated and calibrated classifiers
lr_uncalibrated = LogisticRegression(random_state=42)
lr_calibrated = CalibratedClassifierCV(lr_uncalibrated, method='sigmoid',

# Fit both
lr_uncalibrated.fit(X_train_binary, y_train_binary)
lr_calibrated.fit(X_train_binary, y_train_binary)

# Get probabilities
prob_uncalibrated = lr_uncalibrated.predict_proba(X_test_binary)[: , 1]
prob_calibrated = lr_calibrated.predict_proba(X_test_binary)[: , 1]

```

```

# Plot calibration curves
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
fraction_of_positives, mean_predicted_value = calibration_curve(y_test_bi
plt.plot(mean_predicted_value, fraction_of_positives, "s-", label="Uncali
plt.plot([0, 1], [0, 1], "k:", label="Perfectly calibrated")
plt.xlabel('Mean Predicted Probability')
plt.ylabel('Fraction of Positives')
plt.title('Calibration Plot - Uncalibrated', fontweight='bold')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
fraction_of_positives, mean_predicted_value = calibration_curve(y_test_bi
plt.plot(mean_predicted_value, fraction_of_positives, "s-", label="Calibr
plt.plot([0, 1], [0, 1], "k:", label="Perfectly calibrated")
plt.xlabel('Mean Predicted Probability')
plt.ylabel('Fraction of Positives')
plt.title('Calibration Plot - Calibrated', fontweight='bold')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# 5. REGULARIZATION
print("\n🚧 5. REGULARIZATION")
print("-" * 25)

# Compare different regularization strengths
C_values = [0.01, 0.1, 1, 10, 100]
train_scores = []
test_scores = []

for C in C_values:
    lr_reg = LogisticRegression(C=C, random_state=42)
    lr_reg.fit(X_train_binary, y_train_binary)
    train_scores.append(lr_reg.score(X_train_binary, y_train_binary))
    test_scores.append(lr_reg.score(X_test_binary, y_test_binary))

# Plot regularization effect
plt.figure(figsize=(10, 6))
plt.plot(C_values, train_scores, 'o-', label='Training Score', color='blue')
plt.plot(C_values, test_scores, 'o-', label='Test Score', color='red')
plt.xscale('log')
plt.xlabel('C (Inverse Regularization Strength)')
plt.ylabel('Accuracy')
plt.title('Effect of Regularization on Logistic Regression', fontweight='bold')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

print("\n✅ Logistic Regression completed!")
print("\n💡 Key Takeaways:")
print("• Logistic regression outputs probabilities, not just predictions")
print("• ROC curve and AUC are important for binary classification")
print("• Regularization helps prevent overfitting")

```

```
print("• Probability calibration improves reliability of predictions")
print("• Multiclass logistic regression uses one-vs-rest or one-vs-one st
```

8. Decision Trees

```
In [ ]: # =====
# 8. DECISION TREES
# =====

print("🌳 DECISION TREES")
print("=" * 20)

print("\n🎯 WHAT ARE DECISION TREES?")
print("• Non-parametric supervised learning algorithm")
print("• Creates a tree-like model of decisions")
print("• Each node represents a feature test")
print("• Each branch represents an outcome")
print("• Each leaf represents a class label or value")

print("\n📊 WHEN TO USE:")
print("• Need interpretable models")
print("• Non-linear relationships")
print("• Mixed data types (numerical and categorical)")
print("• Feature importance is important")
print("• Can handle missing values")

# 1. DECISION TREE CLASSIFIER
print("\n🌳 1. DECISION TREE CLASSIFIER")
print("-" * 40)

# Generate classification data
X_tree, y_tree = make_classification(n_samples=200, n_features=2, n_redundant=0,
                                    n_informative=2, n_clusters_per_class=2,
                                    random_state=42)

# Split data
X_train_tree, X_test_tree, y_train_tree, y_test_tree = train_test_split(
    X_tree, y_tree, test_size=0.2, random_state=42, stratify=y_tree)

# Create and train decision tree
dt_classifier = DecisionTreeClassifier(random_state=42, max_depth=3)
dt_classifier.fit(X_train_tree, y_train_tree)

# Make predictions
y_pred_tree = dt_classifier.predict(X_test_tree)
y_pred_proba_tree = dt_classifier.predict_proba(X_test_tree)

# Calculate metrics
accuracy_tree = accuracy_score(y_test_tree, y_pred_tree)
precision_tree = precision_score(y_test_tree, y_pred_tree)
recall_tree = recall_score(y_test_tree, y_pred_tree)
f1_tree = f1_score(y_test_tree, y_pred_tree)

print(f"\n📊 Results:")
print(f"    Accuracy: {accuracy_tree:.3f}")
print(f"    Precision: {precision_tree:.3f}")
print(f"    Recall: {recall_tree:.3f}")
print(f"    F1-Score: {f1_tree:.3f}")
```

```

# Visualize decision tree
plt.figure(figsize=(15, 10))
plot_tree(dt_classifier, feature_names=['Feature 1', 'Feature 2'],
          class_names=['Class 0', 'Class 1'], filled=True, rounded=True)
plt.title('Decision Tree Visualization', fontsize=16, fontweight='bold')
plt.show()

# Visualize decision boundary
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

# Decision boundary
DecisionBoundaryDisplay.from_estimator(dt_classifier, X_tree, ax=axes[0],
axes[0].scatter(X_tree[:, 0], X_tree[:, 1], c=y_tree, cmap='viridis', edg
axes[0].set_title('Decision Tree Decision Boundary', fontweight='bold')
axes[0].set_xlabel('Feature 1')
axes[0].set_ylabel('Feature 2')

# Feature importance
feature_importance = dt_classifier.feature_importances_
axes[1].bar(['Feature 1', 'Feature 2'], feature_importance, color='skyblu
axes[1].set_title('Feature Importance', fontweight='bold')
axes[1].set_ylabel('Importance')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# 2. DECISION TREE REGRESSOR
print("\n📌 2. DECISION TREE REGRESSOR")
print("-" * 40)

# Generate regression data
X_reg_tree, y_reg_tree = make_regression(n_samples=200, n_features=1, noi

# Split data
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg_tree, y_reg_tree, test_size=0.2, random_state=42)

# Create and train decision tree regressor
dt_regressor = DecisionTreeRegressor(random_state=42, max_depth=3)
dt_regressor.fit(X_train_reg, y_train_reg)

# Make predictions
y_pred_reg = dt_regressor.predict(X_test_reg)

# Calculate metrics
mse_reg = mean_squared_error(y_test_reg, y_pred_reg)
r2_reg = r2_score(y_test_reg, y_pred_reg)
mae_reg = mean_absolute_error(y_test_reg, y_pred_reg)

print(f"\n📊 Results:")
print(f"    Mean Squared Error: {mse_reg:.3f}")
print(f"    R-squared: {r2_reg:.3f}")
print(f"    Mean Absolute Error: {mae_reg:.3f}")

# Visualize regression tree
plt.figure(figsize=(15, 10))
plot_tree(dt_regressor, feature_names=['Feature'], filled=True, rounded=T
plt.title('Decision Tree Regressor Visualization', fontsize=16, fontweigh

```

```

plt.show()

# Visualize regression results
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
X_sorted = np.sort(X_test_reg, axis=0)
y_pred_sorted = dt_regressor.predict(X_sorted)
plt.scatter(X_test_reg, y_test_reg, alpha=0.6, color='blue', label='Actual')
plt.plot(X_sorted, y_pred_sorted, color='red', linewidth=2, label='Predicted')
plt.title('Decision Tree Regression', fontweight='bold')
plt.xlabel('Feature')
plt.ylabel('Target')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.scatter(y_test_reg, y_pred_reg, alpha=0.6)
plt.plot([y_test_reg.min(), y_test_reg.max()], [y_test_reg.min(), y_test_reg.max()], color='red', linewidth=2)
plt.title('Actual vs Predicted', fontweight='bold')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# 3. TREE PRUNING AND OVERFITTING
print("\n✂ 3. TREE PRUNING AND OVERFITTING")
print("-" * 40)

# Test different max_depth values
max_depths = range(1, 11)
train_scores = []
test_scores = []

for depth in max_depths:
    dt_depth = DecisionTreeClassifier(random_state=42, max_depth=depth)
    dt_depth.fit(X_train_tree, y_train_tree)
    train_scores.append(dt_depth.score(X_train_tree, y_train_tree))
    test_scores.append(dt_depth.score(X_test_tree, y_test_tree))

# Plot overfitting demonstration
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(max_depths, train_scores, 'o-', label='Training Score', color='blue')
plt.plot(max_depths, test_scores, 'o-', label='Test Score', color='red')
plt.xlabel('Max Depth')
plt.ylabel('Accuracy')
plt.title('Overfitting in Decision Trees', fontweight='bold')
plt.legend()
plt.grid(True, alpha=0.3)

# Show tree with different depths
plt.subplot(1, 2, 2)
dt_shallow = DecisionTreeClassifier(random_state=42, max_depth=2)
dt_deep = DecisionTreeClassifier(random_state=42, max_depth=10)
dt_shallow.fit(X_train_tree, y_train_tree)
dt_deep.fit(X_train_tree, y_train_tree)

```

```

# Compare decision boundaries
DecisionBoundaryDisplay.from_estimator(dt_shallow, X_tree, ax=plt.gca(),
plt.scatter(X_tree[:, 0], X_tree[:, 1], c=y_tree, cmap='viridis', edgecol
plt.title('Shallow Tree (max_depth=2)', fontweight='bold')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')

plt.tight_layout()
plt.show()

# 4. FEATURE IMPORTANCE
print("\n★ 4. FEATURE IMPORTANCE")
print("-" * 30)

# Use a dataset with more features
X_multi_tree, y_multi_tree = make_classification(n_samples=200, n_feature
n_informative=5, random_

# Split data
X_train_multi, X_test_multi, y_train_multi, y_test_multi = train_test_spl
X_multi_tree, y_multi_tree, test_size=0.2, random_state=42, stratify=

# Train decision tree
dt_multi = DecisionTreeClassifier(random_state=42, max_depth=4)
dt_multi.fit(X_train_multi, y_train_multi)

# Get feature importance
feature_importance = dt_multi.feature_importances_
feature_names = [f'Feature {i+1}' for i in range(X_multi_tree.shape[1])]

# Create importance dataframe
importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': feature_importance
}).sort_values('Importance', ascending=True)

# Visualize feature importance
plt.figure(figsize=(10, 6))
plt.barh(importance_df['Feature'], importance_df['Importance'], color='li
plt.title('Feature Importance in Decision Tree', fontweight='bold')
plt.xlabel('Importance')
plt.grid(True, alpha=0.3)
plt.show()

print("\n📊 Feature Importance:")
for i, (feature, importance) in enumerate(zip(importance_df['Feature'], i
    print(f"    {feature}: {importance:.3f}")

# 5. DECISION TREE ADVANTAGES AND DISADVANTAGES
print("\n✅❌ 5. ADVANTAGES AND DISADVANTAGES")
print("-" * 45)

print("\n✅ ADVANTAGES:")
print("• Easy to understand and interpret")
print("• Can handle both numerical and categorical data")
print("• Requires little data preparation")
print("• Can handle non-linear relationships")
print("• Provides feature importance")
print("• Can handle missing values")

```

```

print("\n❌ DISADVANTAGES:")
print("• Prone to overfitting")
print("• Can be unstable (small changes in data can lead to different trees)")
print("• Biased towards features with more levels")
print("• Can create biased trees if some classes dominate")
print("• Not suitable for continuous output prediction")

# 6. RANDOM FOREST (ENSEMBLE OF DECISION TREES)
print("\n🌲 6. RANDOM FOREST")
print("-" * 25)

# Create random forest
rf = RandomForestClassifier(n_estimators=100, random_state=42, max_depth=3)
rf.fit(X_train_multi, y_train_multi)

# Make predictions
y_pred_rf = rf.predict(X_test_multi)
accuracy_rf = accuracy_score(y_test_multi, y_pred_rf)

print(f"\n📊 Random Forest Results:")
print(f"    Accuracy: {accuracy_rf:.3f}")
print(f"    Number of trees: {rf.n_estimators}")

# Compare single tree vs random forest
dt_single = DecisionTreeClassifier(random_state=42, max_depth=3)
dt_single.fit(X_train_multi, y_train_multi)
accuracy_dt = accuracy_score(y_test_multi, dt_single.predict(X_test_multi))

print(f"\n📊 Comparison:")
print(f"    Single Decision Tree: {accuracy_dt:.3f}")
print(f"    Random Forest: {accuracy_rf:.3f}")
print(f"    Improvement: {accuracy_rf - accuracy_dt:.3f}")

print("\n✅ Decision Trees completed!")
print("\n💡 Key Takeaways:")
print("• Decision trees are interpretable and easy to understand")
print("• Prone to overfitting – use pruning and ensemble methods")
print("• Feature importance helps with feature selection")
print("• Random Forest reduces overfitting and improves performance")
print("• Good for both classification and regression tasks")

```

9. Model Evaluation Metrics

```

In [ ]: # =====
# 9. MODEL EVALUATION METRICS
# =====

print("📊 MODEL EVALUATION METRICS")
print("-" * 35)

print("\n🎯 WHY EVALUATE MODELS?")
print("• Measure model performance")
print("• Compare different models")
print("• Identify areas for improvement")
print("• Ensure model generalizes well")
print("• Make informed decisions about deployment")

```

```

# 1. CLASSIFICATION METRICS
print("\n📊 1. CLASSIFICATION METRICS")
print("-" * 35)

# Generate classification data
X_eval, y_eval = make_classification(n_samples=1000, n_features=4, n_redu
                                   n_informative=4, n_classes=2, random_

# Split data
X_train_eval, X_test_eval, y_train_eval, y_test_eval = train_test_split(
    X_eval, y_eval, test_size=0.2, random_state=42, stratify=y_eval)

# Train multiple models for comparison
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC

models = {
    'Logistic Regression': LogisticRegression(random_state=42),
    'Decision Tree': DecisionTreeClassifier(random_state=42, max_depth=5),
    'Random Forest': RandomForestClassifier(random_state=42, n_estimators
    'SVM': SVC(random_state=42, probability=True)
}

# Train and evaluate models
results = {}
for name, model in models.items():
    model.fit(X_train_eval, y_train_eval)
    y_pred = model.predict(X_test_eval)
    y_pred_proba = model.predict_proba(X_test_eval)[:, 1] if hasattr(model, 'predict_proba') else None

    results[name] = {
        'y_pred': y_pred,
        'y_pred_proba': y_pred_proba,
        'accuracy': accuracy_score(y_test_eval, y_pred),
        'precision': precision_score(y_test_eval, y_pred),
        'recall': recall_score(y_test_eval, y_pred),
        'f1': f1_score(y_test_eval, y_pred)
    }

# Display results
print("\n📋 Model Comparison:")
print("-" * 50)
print(f"{'Model':<20} {'Accuracy':<10} {'Precision':<10} {'Recall':<10} {'F1':<10}")
print("-" * 50)
for name, metrics in results.items():
    print(f"{'name':<20} {'metrics['accuracy']':<10.3f} {'metrics['precision']':<10.3f} {'metrics['recall']':<10.3f} {'metrics['f1']':<10.3f}")

# 2. CONFUSION MATRIX
print("\n📊 2. CONFUSION MATRIX")
print("-" * 30)

# Use the best performing model
best_model_name = max(results.keys(), key=lambda x: results[x]['accuracy'])
best_model = models[best_model_name]
y_pred_best = results[best_model_name]['y_pred']

# Create confusion matrix
cm = confusion_matrix(y_test_eval, y_pred_best)

```



```

# Visualize confusion matrix
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

# Raw confusion matrix
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0])
axes[0].set_title(f'Confusion Matrix - {best_model_name}', fontweight='bold')
axes[0].set_xlabel('Predicted')
axes[0].set_ylabel('Actual')

# Normalized confusion matrix
cm_normalized = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
sns.heatmap(cm_normalized, annot=True, fmt='.2f', cmap='Blues', ax=axes[1])
axes[1].set_title(f'Normalized Confusion Matrix - {best_model_name}', fontweight='bold')
axes[1].set_xlabel('Predicted')
axes[1].set_ylabel('Actual')

plt.tight_layout()
plt.show()

# 3. ROC CURVE AND AUC
print("\n📊 3. ROC CURVE AND AUC")
print("-" * 30)

# Plot ROC curves for all models
plt.figure(figsize=(12, 8))

for name, metrics in results.items():
    if metrics['y_pred_proba'] is not None:
        fpr, tpr, _ = roc_curve(y_test_eval, metrics['y_pred_proba'])
        auc = roc_auc_score(y_test_eval, metrics['y_pred_proba'])
        plt.plot(fpr, tpr, label=f'{name} (AUC = {auc:.3f})', linewidth=2)

plt.plot([0, 1], [0, 1], 'k--', label='Random Classifier')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves Comparison', fontweight='bold')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

# 4. PRECISION-RECALL CURVE
print("\n📊 4. PRECISION-RECALL CURVE")
print("-" * 35)

from sklearn.metrics import precision_recall_curve, average_precision_score

plt.figure(figsize=(12, 8))

for name, metrics in results.items():
    if metrics['y_pred_proba'] is not None:
        precision, recall, _ = precision_recall_curve(y_test_eval, metrics['y_pred_proba'])
        ap = average_precision_score(y_test_eval, metrics['y_pred_proba'])
        plt.plot(recall, precision, label=f'{name} (AP = {ap:.3f})', line

# Baseline (random classifier)
baseline = len(y_test_eval[y_test_eval == 1]) / len(y_test_eval)
plt.axhline(y=baseline, color='k', linestyle='--', label=f'Random Classif

```

```

plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curves', fontweight='bold')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

# 5. REGRESSION METRICS
print("\n📊 5. REGRESSION METRICS")
print("-" * 30)

# Generate regression data
X_reg_eval, y_reg_eval = make_regression(n_samples=1000, n_features=4, no

# Split data
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg_eval, y_reg_eval, test_size=0.2, random_state=42)

# Train regression models
from sklearn.ensemble import RandomForestRegressor
from sklearn.svm import SVR

reg_models = {
    'Linear Regression': LinearRegression(),
    'Decision Tree': DecisionTreeRegressor(random_state=42, max_depth=5),
    'Random Forest': RandomForestRegressor(random_state=42, n_estimators=
    'SVR': SVR()
}

reg_results = {}
for name, model in reg_models.items():
    model.fit(X_train_reg, y_train_reg)
    y_pred = model.predict(X_test_reg)

    reg_results[name] = {
        'y_pred': y_pred,
        'mse': mean_squared_error(y_test_reg, y_pred),
        'rmse': np.sqrt(mean_squared_error(y_test_reg, y_pred)),
        'mae': mean_absolute_error(y_test_reg, y_pred),
        'r2': r2_score(y_test_reg, y_pred)
    }

# Display regression results
print("\n📊 Regression Model Comparison:")
print("-" * 60)
print(f"{'Model':<20} {'MSE':<10} {'RMSE':<10} {'MAE':<10} {'R²':<10}")
print("-" * 60)
for name, metrics in reg_results.items():
    print(f"{'name':<20} {'metrics['mse']':<10.3f} {'metrics['rmse']':<10.3f} "
          f"{'metrics['mae']':<10.3f} {'metrics['r2']':<10.3f}")

# Visualize regression results
fig, axes = plt.subplots(2, 2, figsize=(15, 12))
axes = axes.flatten()

for i, (name, metrics) in enumerate(reg_results.items()):
    axes[i].scatter(y_test_reg, metrics['y_pred'], alpha=0.6)
    axes[i].plot([y_test_reg.min(), y_test_reg.max()],

```

```

        [y_test_reg.min(), y_test_reg.max()], 'r--', lw=2)
    axes[i].set_xlabel('Actual Values')
    axes[i].set_ylabel('Predicted Values')
    axes[i].set_title(f'{name} (R² = {metrics["r2"]:.3f})', fontweight='b')
    axes[i].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# 6. CROSS-VALIDATION SCORES
print("\n📊 6. CROSS-VALIDATION SCORES")
print("-" * 35)

# Perform cross-validation on classification models
cv_scores = {}
for name, model in models.items():
    scores = cross_val_score(model, X_eval, y_eval, cv=5, scoring='accuracy')
    cv_scores[name] = scores

# Display CV results
print("\n📊 Cross-Validation Results:")
print("-" * 40)
for name, scores in cv_scores.items():
    print(f'{name:<20}: {scores.mean():.3f} (+/- {scores.std() * 2:.3f})')

# Visualize CV results
plt.figure(figsize=(12, 6))
plt.boxplot([cv_scores[name] for name in cv_scores.keys()],
            labels=list(cv_scores.keys()))
plt.title('Cross-Validation Scores Distribution', fontweight='bold')
plt.ylabel('Accuracy')
plt.xticks(rotation=45)
plt.grid(True, alpha=0.3)
plt.show()

# 7. LEARNING CURVES
print("\n📈 7. LEARNING CURVES")
print("-" * 25)

from sklearn.model_selection import learning_curve

# Generate learning curves for the best model
train_sizes, train_scores, val_scores = learning_curve(
    best_model, X_eval, y_eval, cv=5, n_jobs=-1,
    train_sizes=np.linspace(0.1, 1.0, 10))

# Calculate mean and std
train_mean = np.mean(train_scores, axis=1)
train_std = np.std(train_scores, axis=1)
val_mean = np.mean(val_scores, axis=1)
val_std = np.std(val_scores, axis=1)

# Plot learning curves
plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_mean, 'o-', color='blue', label='Training Score')
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, color='blue')
plt.plot(train_sizes, val_mean, 'o-', color='red', label='Validation Score')
plt.fill_between(train_sizes, val_mean - val_std, val_mean + val_std, color='red')
plt.xlabel('Training Set Size')
plt.ylabel('Accuracy')

```

```

plt.title(f'Learning Curves - {best_model_name}', fontweight='bold')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

print("\n✅ Model Evaluation completed!")
print("\n💡 Key Takeaways:")
print("• Use multiple metrics to get a complete picture")
print("• Cross-validation provides more robust estimates")
print("• Learning curves help identify bias vs variance")
print("• ROC curves are great for binary classification")
print("• Precision-Recall curves are better for imbalanced datasets")
print("• Always validate on unseen test data")

```

10. Overfitting and Underfitting

```

In [ ]: # =====
# 10. OVERFITTING AND UNDERFITTING
# =====

print("⚖️ OVERFITTING AND UNDERFITTING")
print("=" * 40)

print("\n🎯 WHAT ARE OVERFITTING AND UNDERFITTING?")
print("• Overfitting: Model performs well on training data but poorly on")
print("• Underfitting: Model is too simple to capture underlying patterns")
print("• Bias-Variance Tradeoff: Balance between model complexity and gen")

print("\n📊 WHEN DO THEY OCCUR?")
print("• Overfitting: Model is too complex, learns noise in training data")
print("• Underfitting: Model is too simple, cannot learn patterns")
print("• Goal: Find the sweet spot between the two")

# 1. DEMONSTRATE OVERFITTING WITH DECISION TREES
print("\n🌳 1. OVERFITTING WITH DECISION TREES")
print("-" * 45)

# Generate data
X_overfit, y_overfit = make_classification(n_samples=200, n_features=2, n
                                         n_informative=2, n_clusters_per
                                         random_state=42)

# Split data
X_train_over, X_test_over, y_train_over, y_test_over = train_test_split(
    X_overfit, y_overfit, test_size=0.3, random_state=42, stratify=y_over

# Test different tree depths
max_depths = range(1, 21)
train_scores = []
test_scores = []

for depth in max_depths:
    dt = DecisionTreeClassifier(random_state=42, max_depth=depth)
    dt.fit(X_train_over, y_train_over)
    train_scores.append(dt.score(X_train_over, y_train_over))
    test_scores.append(dt.score(X_test_over, y_test_over))

# Find optimal depth

```

```

optimal_depth = max_depths[np.argmax(test_scores)]

# Plot overfitting demonstration
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.plot(max_depths, train_scores, 'o-', label='Training Score', color='b')
plt.plot(max_depths, test_scores, 'o-', label='Test Score', color='red',)
plt.axvline(x=optimal_depth, color='green', linestyle='--', label=f'Optim')
plt.xlabel('Max Depth')
plt.ylabel('Accuracy')
plt.title('Overfitting in Decision Trees', fontweight='bold')
plt.legend()
plt.grid(True, alpha=0.3)

# Show decision boundaries for different depths
depths_to_show = [1, 3, 10]
for i, depth in enumerate(depths_to_show):
    plt.subplot(1, 3, i+2)
    dt_show = DecisionTreeClassifier(random_state=42, max_depth=depth)
    dt_show.fit(X_train_over, y_train_over)

    DecisionBoundaryDisplay.from_estimator(dt_show, X_overfit, ax=plt.gca)
    plt.scatter(X_overfit[:, 0], X_overfit[:, 1], c=y_overfit, cmap='viridis')
    plt.title(f'Max Depth = {depth}\\nTrain: {dt_show.score(X_train_over, y_train_over):.3f}')
    plt.xlabel('Feature 1')
    plt.ylabel('Feature 2')

plt.tight_layout()
plt.show()

print(f"\n📊 Results:")
print(f"    Optimal depth: {optimal_depth}")
print(f"    Best training score: {max(train_scores):.3f}")
print(f"    Best test score: {max(test_scores):.3f}")
print(f"    Overfitting gap: {max(train_scores) - max(test_scores):.3f}")

# 2. DEMONSTRATE UNDERFITTING WITH LINEAR REGRESSION
print("\n📉 2. UNDERFITTING WITH LINEAR REGRESSION")
print("-" * 50)

# Generate non-linear data
X_underfit = np.linspace(-3, 3, 100).reshape(-1, 1)
y_underfit = 0.5 * X_underfit.flatten()**3 - 2 * X_underfit.flatten()**2

# Split data
X_train_under, X_test_under, y_train_under, y_test_under = train_test_split(X_underfit, y_underfit, test_size=0.3, random_state=42)

# Test different polynomial degrees
degrees = [1, 2, 3, 4, 5, 6]
train_scores_under = []
test_scores_under = []

for degree in degrees:
    poly_features = PolynomialFeatures(degree=degree)
    X_poly = poly_features.fit_transform(X_train_under)
    X_test_poly = poly_features.transform(X_test_under)

```

```

lr = LinearRegression()
lr.fit(X_poly, y_train_under)

train_scores_under.append(lr.score(X_poly, y_train_under))
test_scores_under.append(lr.score(X_test_poly, y_test_under))

# Plot underfitting demonstration
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.plot(degrees, train_scores_under, 'o-', label='Training Score', color='blue')
plt.plot(degrees, test_scores_under, 'o-', label='Test Score', color='red')
plt.xlabel('Polynomial Degree')
plt.ylabel('R2 Score')
plt.title('Underfitting vs Overfitting', fontweight='bold')
plt.legend()
plt.grid(True, alpha=0.3)

# Show fits for different degrees
degrees_to_show = [1, 3, 6]
for i, degree in enumerate(degrees_to_show):
    plt.subplot(1, 3, i+2)

    poly_features = PolynomialFeatures(degree=degree)
    X_poly = poly_features.fit_transform(X_underfit)

    lr = LinearRegression()
    lr.fit(X_poly, y_underfit)
    y_pred = lr.predict(X_poly)

    plt.scatter(X_underfit, y_underfit, alpha=0.6, color='blue', label='Data')
    plt.plot(X_underfit, y_pred, color='red', linewidth=2, label='Fit')
    plt.title(f'Degree = {degree} \n R2 = {lr.score(X_poly, y_underfit):.3}')
    plt.xlabel('X')
    plt.ylabel('y')
    plt.legend()

plt.tight_layout()
plt.show()

# 3. BIAS-VARIANCE DECOMPOSITION
print("\n📊 3. BIAS-VARIANCE DECOMPOSITION")
print("-" * 40)

# Generate data for bias-variance analysis
X_bv, y_bv = make_regression(n_samples=100, n_features=1, noise=10, random_state=42)

# Create multiple datasets by resampling
n_datasets = 50
models = []
predictions = []

for i in range(n_datasets):
    # Bootstrap sample
    indices = np.random.choice(len(X_bv), size=len(X_bv), replace=True)
    X_sample = X_bv[indices]
    y_sample = y_bv[indices]

    # Train model
    model = LinearRegression()

```

```

model.fit(X_sample, y_sample)
models.append(model)

# Make predictions on original data
pred = model.predict(X_bv)
predictions.append(pred)

# Calculate bias and variance
predictions = np.array(predictions)
mean_prediction = np.mean(predictions, axis=0)
bias_squared = np.mean((mean_prediction - y_bv) ** 2)
variance = np.mean(np.var(predictions, axis=0))
noise = np.var(y_bv - mean_prediction)

print(f"\n📊 Bias-Variance Analysis:")
print(f"    Bias²: {bias_squared:.3f}")
print(f"    Variance: {variance:.3f}")
print(f"    Noise: {noise:.3f}")
print(f"    Total Error: {bias_squared + variance + noise:.3f}")

# Visualize bias-variance
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
for i in range(min(10, n_datasets)):
    plt.plot(X_bv, predictions[i], alpha=0.3, color='blue')
plt.plot(X_bv, mean_prediction, color='red', linewidth=3, label='Average')
plt.scatter(X_bv, y_bv, color='black', alpha=0.6, label='True Data')
plt.title('Model Predictions (Bias-Variance)', fontweight='bold')
plt.xlabel('X')
plt.ylabel('y')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.bar(['Bias²', 'Variance', 'Noise'], [bias_squared, variance, noise],
        color=['red', 'blue', 'green'], alpha=0.7)
plt.title('Error Components', fontweight='bold')
plt.ylabel('Error')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# 4. REGULARIZATION TO PREVENT OVERFITTING
print("\n🏗️ 4. REGULARIZATION TECHNIQUES")
print("-" * 40)

# Ridge Regression (L2 regularization)
from sklearn.linear_model import Ridge

# Test different regularization strengths
alphas = np.logspace(-4, 4, 50)
ridge_scores = []

for alpha in alphas:
    ridge = Ridge(alpha=alpha)
    ridge.fit(X_train_over, y_train_over)
    ridge_scores.append(ridge.score(X_test_over, y_test_over))

```

```

# Lasso Regression (L1 regularization)
from sklearn.linear_model import Lasso

lasso_scores = []
for alpha in alphas:
    lasso = Lasso(alpha=alpha, max_iter=1000)
    lasso.fit(X_train_over, y_train_over)
    lasso_scores.append(lasso.score(X_test_over, y_test_over))

# Plot regularization effects
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.semilogx(alphas, ridge_scores, 'b-', label='Ridge')
plt.semilogx(alphas, lasso_scores, 'r-', label='Lasso')
plt.xlabel('Regularization Strength ( $\alpha$ )')
plt.ylabel('R2 Score')
plt.title('Regularization Effect', fontweight='bold')
plt.legend()
plt.grid(True, alpha=0.3)

# Show coefficient paths
plt.subplot(1, 3, 2)
coef_paths = []
for alpha in alphas[::5]: # Sample every 5th alpha
    ridge = Ridge(alpha=alpha)
    ridge.fit(X_train_over, y_train_over)
    coef_paths.append(ridge.coef_)

coef_paths = np.array(coef_paths)
for i in range(coef_paths.shape[1]):
    plt.semilogx(alphas[::5], coef_paths[:, i], label=f'Feature {i+1}')
plt.xlabel('Regularization Strength ( $\alpha$ )')
plt.ylabel('Coefficient Value')
plt.title('Ridge Coefficient Paths', fontweight='bold')
plt.legend()
plt.grid(True, alpha=0.3)

# Cross-validation for optimal alpha
from sklearn.linear_model import RidgeCV

ridge_cv = RidgeCV(alphas=alphas, cv=5)
ridge_cv.fit(X_train_over, y_train_over)

plt.subplot(1, 3, 3)
plt.semilogx(alphas, ridge_scores, 'b-', label='Ridge')
plt.axvline(x=ridge_cv.alpha_, color='red', linestyle='--', label=f'Optim')
plt.xlabel('Regularization Strength ( $\alpha$ )')
plt.ylabel('R2 Score')
plt.title('Optimal Regularization', fontweight='bold')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print(f"\n📊 Regularization Results:")
print(f"    Optimal Ridge  $\alpha$ : {ridge_cv.alpha_:.3f}")
print(f"    Best Ridge R2: {max(ridge_scores):.3f}")

```


5. VALIDATION CURVES

```
print("\n📄 5. VALIDATION CURVES")
```

```
print("-" * 25)
```

```
from sklearn.model_selection import validation_curve
```

```
# Generate validation curves for different parameters
```

```
param_range = np.logspace(-4, 2, 20)
```

```
train_scores_val, test_scores_val = validation_curve(  
    Ridge(), X_overfit, y_overfit, param_name='alpha', param_range=param_  
    cv=5, scoring='r2', n_jobs=-1)
```

```
# Calculate mean and std
```

```
train_mean = np.mean(train_scores_val, axis=1)
```

```
train_std = np.std(train_scores_val, axis=1)
```

```
test_mean = np.mean(test_scores_val, axis=1)
```

```
test_std = np.std(test_scores_val, axis=1)
```

```
# Plot validation curves
```

```
plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 2, 1)
```

```
plt.semilogx(param_range, train_mean, 'o-', color='blue', label='Training
```

```
plt.fill_between(param_range, train_mean - train_std, train_mean + train_
```

```
plt.semilogx(param_range, test_mean, 'o-', color='red', label='Validation
```

```
plt.fill_between(param_range, test_mean - test_std, test_mean + test_std,
```

```
plt.xlabel('Regularization Strength ( $\alpha$ )')
```

```
plt.ylabel('R2 Score')
```

```
plt.title('Validation Curves - Ridge Regression', fontweight='bold')
```

```
plt.legend()
```

```
plt.grid(True, alpha=0.3)
```

```
# Learning curves for different sample sizes
```

```
plt.subplot(1, 2, 2)
```

```
train_sizes, train_scores_lc, val_scores_lc = learning_curve(  
    Ridge(alpha=0.1), X_overfit, y_overfit, cv=5, n_jobs=-1,
```

```
    train_sizes=np.linspace(0.1, 1.0, 10))
```

```
train_mean_lc = np.mean(train_scores_lc, axis=1)
```

```
train_std_lc = np.std(train_scores_lc, axis=1)
```

```
val_mean_lc = np.mean(val_scores_lc, axis=1)
```

```
val_std_lc = np.std(val_scores_lc, axis=1)
```

```
plt.plot(train_sizes, train_mean_lc, 'o-', color='blue', label='Training
```

```
plt.fill_between(train_sizes, train_mean_lc - train_std_lc, train_mean_lc
```

```
plt.plot(train_sizes, val_mean_lc, 'o-', color='red', label='Validation S
```

```
plt.fill_between(train_sizes, val_mean_lc - val_std_lc, val_mean_lc + val
```

```
plt.xlabel('Training Set Size')
```

```
plt.ylabel('R2 Score')
```

```
plt.title('Learning Curves - Ridge Regression', fontweight='bold')
```

```
plt.legend()
```

```
plt.grid(True, alpha=0.3)
```

```
plt.tight_layout()
```

```
plt.show()
```

6. PREVENTION STRATEGIES

```
print("\n📄 6. PREVENTION STRATEGIES")
```

```
print("-" * 35)
```

```

print("\n✅ PREVENTING OVERFITTING:")
print("• Use cross-validation")
print("• Apply regularization (L1, L2)")
print("• Reduce model complexity")
print("• Increase training data")
print("• Use ensemble methods")
print("• Early stopping")
print("• Dropout (for neural networks)")

print("\n✅ PREVENTING UNDERFITTING:")
print("• Increase model complexity")
print("• Add more features")
print("• Reduce regularization")
print("• Use more sophisticated algorithms")
print("• Feature engineering")

print("\n✅ GENERAL STRATEGIES:")
print("• Always use train/validation/test splits")
print("• Monitor training and validation performance")
print("• Use learning curves to diagnose issues")
print("• Try different algorithms")
print("• Collect more data if possible")

print("\n✅ Overfitting and Underfitting completed!")
print("\n💡 Key Takeaways:")
print("• Overfitting: High training performance, low test performance")
print("• Underfitting: Low performance on both training and test")
print("• Use validation curves to find optimal parameters")
print("• Regularization helps prevent overfitting")
print("• Learning curves help diagnose bias vs variance")
print("• Always validate on unseen data")

```

11. Feature Selection Techniques

In []:

```

# =====
# 11. FEATURE SELECTION TECHNIQUES
# =====

print("🔍 FEATURE SELECTION TECHNIQUES")
print("=" * 35)

print("\n🎯 WHY SELECT FEATURES?")
print("• Reduce overfitting")
print("• Improve model performance")
print("• Reduce training time")
print("• Improve interpretability")
print("• Remove irrelevant or redundant features")

print("\n📊 TYPES OF FEATURE SELECTION:")
print("• Filter methods: Statistical tests")
print("• Wrapper methods: Model-based selection")
print("• Embedded methods: Built into model training")
print("• Univariate vs Multivariate")

# 1. UNIVARIATE FEATURE SELECTION
print("\n📊 1. UNIVARIATE FEATURE SELECTION")
print("-" * 40)

```

```

# Generate dataset with many features
X_fs, y_fs = make_classification(n_samples=1000, n_features=20, n_redundant=10, n_informative=10, n_clusters_per_class=1,

# Split data
X_train_fs, X_test_fs, y_train_fs, y_test_fs = train_test_split(
    X_fs, y_fs, test_size=0.2, random_state=42, stratify=y_fs)

print(f"\n📁 Dataset: {X_fs.shape[0]} samples, {X_fs.shape[1]} features")

# SelectKBest with f_classif
selector_kbest = SelectKBest(score_func=f_classif, k=10)
X_selected_kbest = selector_kbest.fit_transform(X_train_fs, y_train_fs)

print(f"\n📁 SelectKBest Results:")
print(f"    Selected features: {X_selected_kbest.shape[1]}")
print(f"    Feature scores: {selector_kbest.scores_}")
print(f"    Selected indices: {selector_kbest.get_support(indices=True)}")

# Visualize feature scores
plt.figure(figsize=(15, 6))

plt.subplot(1, 2, 1)
feature_scores = selector_kbest.scores_
feature_indices = np.argsort(feature_scores)[::-1]
plt.bar(range(len(feature_scores)), feature_scores[feature_indices], color='lightgreen')
plt.title('Feature Scores (F-statistic)', fontweight='bold')
plt.xlabel('Feature Rank')
plt.ylabel('F-score')
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
selected_mask = selector_kbest.get_support()
plt.bar(range(len(selected_mask)), selected_mask, color='lightgreen')
plt.title('Selected Features (Top 10)', fontweight='bold')
plt.xlabel('Feature Index')
plt.ylabel('Selected (1) or Not (0)')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# 2. RECURSIVE FEATURE ELIMINATION
print("\n📁 2. RECURSIVE FEATURE ELIMINATION")
print("-" * 45)

# RFE with Logistic Regression
rfe = RFE(estimator=LogisticRegression(random_state=42), n_features_to_select=10)
rfe.fit(X_train_fs, y_train_fs)

X_selected_rfe = rfe.transform(X_train_fs)

print(f"\n📁 RFE Results:")
print(f"    Selected features: {X_selected_rfe.shape[1]}")
print(f"    Feature rankings: {rfe.ranking_}")
print(f"    Selected indices: {rfe.get_support(indices=True)}")

# Visualize RFE results
plt.figure(figsize=(12, 5))

```

```

plt.subplot(1, 2, 1)
plt.bar(range(len(rfe.ranking_)), rfe.ranking_, color='coral')
plt.title('Feature Rankings (RFE)', fontweight='bold')
plt.xlabel('Feature Index')
plt.ylabel('Ranking (1 = Best)')
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
selected_mask_rfe = rfe.get_support()
plt.bar(range(len(selected_mask_rfe)), selected_mask_rfe, color='lightcoral')
plt.title('Selected Features (RFE)', fontweight='bold')
plt.xlabel('Feature Index')
plt.ylabel('Selected (1) or Not (0)')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# 3. EMBEDDED METHODS
print("\n 3. EMBEDDED METHODS")
print("-" * 25)

# L1 Regularization (Lasso)
from sklearn.linear_model import LassoCV

lasso_cv = LassoCV(cv=5, random_state=42)
lasso_cv.fit(X_train_fs, y_train_fs)

# Select features with non-zero coefficients
lasso_mask = lasso_cv.coef_ != 0
X_selected_lasso = X_train_fs[:, lasso_mask]

print(f"\n📊 Lasso Results:")
print(f"    Selected features: {np.sum(lasso_mask)}")
print(f"    Optimal alpha: {lasso_cv.alpha_:.6f}")
print(f"    Selected indices: {np.where(lasso_mask)[0]}")

# Random Forest Feature Importance
rf_selector = RandomForestClassifier(n_estimators=100, random_state=42)
rf_selector.fit(X_train_fs, y_train_fs)

# Select features based on importance threshold
importance_threshold = 0.01
rf_mask = rf_selector.feature_importances_ > importance_threshold
X_selected_rf = X_train_fs[:, rf_mask]

print(f"\n🌲 Random Forest Results:")
print(f"    Selected features: {np.sum(rf_mask)}")
print(f"    Importance threshold: {importance_threshold}")
print(f"    Selected indices: {np.where(rf_mask)[0]}")

# Visualize embedded methods
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.bar(range(len(lasso_cv.coef_)), lasso_cv.coef_, color='orange')
plt.title('Lasso Coefficients', fontweight='bold')
plt.xlabel('Feature Index')
plt.ylabel('Coefficient Value')
plt.grid(True, alpha=0.3)

```

```

plt.subplot(1, 3, 2)
plt.bar(range(len(rf_selector.feature_importances_)), rf_selector.feature
plt.axhline(y=importance_threshold, color='red', linestyle='--', label=f'
plt.title('Random Forest Feature Importance', fontweight='bold')
plt.xlabel('Feature Index')
plt.ylabel('Importance')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(1, 3, 3)
methods = ['SelectKBest', 'RFE', 'Lasso', 'Random Forest']
selected_counts = [X_selected_kbest.shape[1], X_selected_rfe.shape[1],
                    np.sum(lasso_mask), np.sum(rf_mask)]
plt.bar(methods, selected_counts, color=['skyblue', 'coral', 'orange', 'g
plt.title('Features Selected by Each Method', fontweight='bold')
plt.ylabel('Number of Features')
plt.xticks(rotation=45)
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# 4. COMPARISON OF METHODS
print("\n 4. COMPARISON OF METHODS")
print("-" * 35)

# Train models with different feature sets
methods_results = {}

# Original features
lr_original = LogisticRegression(random_state=42)
lr_original.fit(X_train_fs, y_train_fs)
methods_results['Original'] = lr_original.score(X_test_fs, y_test_fs)

# SelectKBest
lr_kbest = LogisticRegression(random_state=42)
lr_kbest.fit(X_selected_kbest, y_train_fs)
X_test_kbest = selector_kbest.transform(X_test_fs)
methods_results['SelectKBest'] = lr_kbest.score(X_test_kbest, y_test_fs)

# RFE
lr_rfe = LogisticRegression(random_state=42)
lr_rfe.fit(X_selected_rfe, y_train_fs)
X_test_rfe = rfe.transform(X_test_fs)
methods_results['RFE'] = lr_rfe.score(X_test_rfe, y_test_fs)

# Lasso
lr_lasso = LogisticRegression(random_state=42)
lr_lasso.fit(X_selected_lasso, y_train_fs)
X_test_lasso = X_test_fs[:, lasso_mask]
methods_results['Lasso'] = lr_lasso.score(X_test_lasso, y_test_fs)

# Random Forest
lr_rf = LogisticRegression(random_state=42)
lr_rf.fit(X_selected_rf, y_train_fs)
X_test_rf = X_test_fs[:, rf_mask]
methods_results['Random Forest'] = lr_rf.score(X_test_rf, y_test_fs)

# Display results

```

```

print("\n🔧 Performance Comparison:")
print("-" * 40)
print(f"{'Method':<15} {'Features':<10} {'Accuracy':<10}")
print("-" * 40)
print(f"{'Original':<15} {X_train_fs.shape[1]:<10} {methods_results['Orig']}<br>
print(f"{'SelectKBest':<15} {X_selected_kbest.shape[1]:<10} {methods_resu<br>
print(f"{'RFE':<15} {X_selected_rfe.shape[1]:<10} {methods_results['RFE']}<br>
print(f"{'Lasso':<15} {np.sum(lasso_mask):<10} {methods_results['Lasso']}<br>
print(f"{'Random Forest':<15} {np.sum(rf_mask):<10} {methods_results['Ran

# Visualize comparison
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
methods = list(methods_results.keys())
accuracies = list(methods_results.values())
plt.bar(methods, accuracies, color=['red', 'skyblue', 'coral', 'orange',
plt.title('Model Performance Comparison', fontweight='bold')
plt.ylabel('Accuracy')
plt.xticks(rotation=45)
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
feature_counts = [X_train_fs.shape[1], X_selected_kbest.shape[1], X_selec<br>
np.sum(lasso_mask), np.sum(rf_mask)]
plt.bar(methods, feature_counts, color=['red', 'skyblue', 'coral', 'orang<br>
plt.title('Number of Features Selected', fontweight='bold')
plt.ylabel('Number of Features')
plt.xticks(rotation=45)
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# 5. FEATURE SELECTION PIPELINE
print("\n🔧 5. FEATURE SELECTION PIPELINE")
print("-" * 40)

# Create a complete pipeline
from sklearn.pipeline import Pipeline

# Pipeline with feature selection
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('selector', SelectKBest(score_func=f_classif, k=10)),
    ('classifier', LogisticRegression(random_state=42))
])

# Fit pipeline
pipeline.fit(X_train_fs, y_train_fs)

# Evaluate pipeline
pipeline_score = pipeline.score(X_test_fs, y_test_fs)

print(f"\n🔧 Pipeline Results:")
print(f"    Pipeline accuracy: {pipeline_score:.3f}")
print(f"    Selected features: {pipeline.named_steps['selector'].get_suppo

# Cross-validation with pipeline
cv_scores_pipeline = cross_val_score(pipeline, X_fs, y_fs, cv=5, scoring=

```

```

print(f"\n📊 Cross-Validation Results:")
print(f"    Mean CV Score: {cv_scores_pipeline.mean():.3f} (+/- {cv_scores

# 6. FEATURE SELECTION BEST PRACTICES
print("\n💡 6. FEATURE SELECTION BEST PRACTICES")
print("-" * 45)

print("\n✅ WHEN TO USE EACH METHOD:")
print("• Filter methods: Quick screening, large datasets")
print("• Wrapper methods: When you have computational resources")
print("• Embedded methods: When using specific algorithms")
print("• Univariate: When features are independent")
print("• Multivariate: When features interact")

print("\n✅ BEST PRACTICES:")
print("• Always use cross-validation")
print("• Consider feature interactions")
print("• Don't over-optimize on validation set")
print("• Consider domain knowledge")
print("• Monitor for data leakage")
print("• Use multiple methods and compare")

print("\n✅ COMMON PITFALLS:")
print("• Selecting features based on test set")
print("• Ignoring feature interactions")
print("• Overfitting to validation set")
print("• Not considering computational cost")
print("• Ignoring business context")

# 7. FEATURE IMPORTANCE ANALYSIS
print("\n⭐ 7. FEATURE IMPORTANCE ANALYSIS")
print("-" * 40)

# Analyze feature importance from different methods
importance_analysis = pd.DataFrame({
    'Feature': [f'Feature_{i}' for i in range(X_fs.shape[1])],
    'F_Score': selector_kbest.scores_,
    'RFE_Ranking': rfe.ranking_,
    'Lasso_Coef': lasso_cv.coef_,
    'RF_Importance': rf_selector.feature_importances_
})

# Normalize scores for comparison
importance_analysis['F_Score_Norm'] = (importance_analysis['F_Score'] - i
importance_analysis['RF_Importance_Norm'] = importance_analysis['RF_Import
importance_analysis['Lasso_Coef_Abs'] = np.abs(importance_analysis['Lasso

# Display top features
print("\n📊 Top 10 Features by Different Methods:")
print("-" * 50)
print(importance_analysis.nlargest(10, 'F_Score_Norm')[['Feature', 'F_Sco

# Visualize feature importance correlation
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.scatter(importance_analysis['F_Score_Norm'], importance_analysis['RF_
plt.xlabel('F-Score (Normalized)')
plt.ylabel('RF Importance (Normalized)')

```

```

plt.title('F-Score vs RF Importance', fontweight='bold')
plt.grid(True, alpha=0.3)

plt.subplot(1, 3, 2)
plt.scatter(importance_analysis['F_Score_Norm'], importance_analysis['Lasso_Coefficient'])
plt.xlabel('F-Score (Normalized)')
plt.ylabel('Lasso |Coefficient|')
plt.title('F-Score vs Lasso Coefficients', fontweight='bold')
plt.grid(True, alpha=0.3)

plt.subplot(1, 3, 3)
plt.scatter(importance_analysis['RF_Importance_Norm'], importance_analysis['Lasso_Coefficient'])
plt.xlabel('RF Importance (Normalized)')
plt.ylabel('Lasso |Coefficient|')
plt.title('RF Importance vs Lasso Coefficients', fontweight='bold')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\n✅ Feature Selection completed!")
print("\n💡 Key Takeaways:")
print("• Feature selection can improve model performance and interpretability")
print("• Different methods work better for different scenarios")
print("• Always validate feature selection with cross-validation")
print("• Consider the trade-off between performance and interpretability")
print("• Use domain knowledge to guide feature selection")
print("• Monitor for overfitting during feature selection")

```

Summary and Next Steps

```

In [ ]: # =====
# SUMMARY AND NEXT STEPS
# =====

print("MODULE 8 COMPLETED!")
print("=" * 25)

print("\nWHAT WE COVERED:")
print("1. Introduction to Machine Learning")
print("    • Supervised, Unsupervised, and Reinforcement Learning")
print("    • Common ML problems and workflow")

print("\n2. Types of Machine Learning Problems")
print("    • Classification, Regression, Clustering, Dimensionality Reduction")
print("    • Visual examples and use cases")

print("\n3. Data Preprocessing and Feature Engineering")
print("    • Handling missing values, categorical variables")
print("    • Feature scaling, engineering, and selection")
print("    • Complete preprocessing pipelines")

print("\n4. Train-Test Split and Cross-Validation")
print("    • Data splitting strategies")
print("    • K-fold and stratified cross-validation")
print("    • Time series splits")

print("\n5. Introduction to scikit-learn")

```



```
print("    • Consistent API, pipelines, model persistence")
print("    • Built-in datasets and visualization tools")

print("\n6. Linear Regression")
print("    • Simple and multiple linear regression")
print("    • Residual analysis, polynomial regression")
print("    • Feature importance and interpretation")

print("\n7. Logistic Regression")
print("    • Binary and multiclass classification")
print("    • ROC curves, probability calibration")
print("    • Regularization techniques")

print("\n8. Decision Trees")
print("    • Classification and regression trees")
print("    • Overfitting, pruning, feature importance")
print("    • Random Forest ensemble")

print("\n9. Model Evaluation Metrics")
print("    • Classification and regression metrics")
print("    • Confusion matrices, ROC curves")
print("    • Cross-validation and learning curves")

print("\n10. Overfitting and Underfitting")
print("    • Bias-variance tradeoff")
print("    • Regularization techniques")
print("    • Validation curves and prevention strategies")

print("\n11. Feature Selection Techniques")
print("    • Filter, wrapper, and embedded methods")
print("    • Performance comparison")
print("    • Best practices and pitfalls")

print("\nNEXT STEPS:")
print("• Practice with real datasets")
print("• Explore ensemble methods (Random Forest, Gradient Boosting)")
print("• Learn about neural networks and deep learning")
print("• Study advanced topics (SVM, Naive Bayes, Clustering)")
print("• Work on end-to-end ML projects")
print("• Learn about model deployment and MLOps")

print("\nKEY TAKEAWAYS:")
print("• Always split your data properly")
print("• Use cross-validation for robust evaluation")
print("• Preprocessing is crucial for model performance")
print("• Start simple, then add complexity")
print("• Visualize your data and results")
print("• Understand the bias-variance tradeoff")
print("• Feature engineering can make a big difference")

print("\nPRACTICAL EXERCISES:")
print("1. Apply these techniques to a real dataset")
print("2. Compare different algorithms on the same data")
print("3. Experiment with feature engineering")
print("4. Build a complete ML pipeline")
print("5. Deploy a model and monitor its performance")

print("\nRECOMMENDED RESOURCES:")
print("• Scikit-learn documentation: https://scikit-learn.org/")
print("• Kaggle Learn: https://www.kaggle.com/learn")
```

```
print("• Hands-On Machine Learning by Aurélien Géron")
print("• The Elements of Statistical Learning by Hastie, Tibshirani, Frie

print("\\nCONGRATULATIONS!")
print("You've completed a comprehensive introduction to Machine Learning!")
print("Keep practicing and building projects to solidify your understandi")
print("\\nHappy Learning!")
```